

FINAL REPORT
KASUS : DATA MART KEMAHASISWAAN
Data Warehouse



Disusun oleh kelompok 4:

1. Adil Aulia Rahma Nurhidayah (122450058)
2. Rosalia Siregar (123450036)
3. Muhammad Hanif Dzaky Arifin (123450064)
4. Haikall Fransisko Simbolon (123450106)

PROGRAM STUDI SAINS DATA
FAKULTAS SAINS
INSTITUT TEKNOLOGI SUMATERA
2025

EXECUTIVE SUMMARY

1. Project Overview

Proyek ini adalah fase puncak dari pembangunan Data Mart Kemahasiswaan ITERA, yang bertujuan mengkonsolidasikan data dari berbagai sistem operasional (SIKAD, Biro Kemahasiswaan, Ormawa) ke dalam satu Data Warehouse terpadu. Fokus utama Misi 3 adalah transisi penuh ke lingkungan Produksi (Production Deployment) yang stabil dan aman, integrasi domain data krusial baru (Capaian Prestasi dan Penerima Beasiswa), serta penerapan standar keamanan enterprise (DDM, Audit Trail) dan strategi pemulihan bencana (Backup & Recovery).

2. Capaian Kunci

Berdasarkan hasil implementasi Data Mart dan proses ETL yang telah dilakukan, sejumlah capaian kunci berhasil diwujudkan dengan dampak langsung terhadap kualitas data dan kemudahan analisis strategis lembaga. Adapun capaian utama tersebut adalah:

- 1) Integrasi Multi-Fact: Data Mart kini berisi empat Fact Table yang terintegrasi penuh: Fact_Partisipasi_Kegiatan (Misi 2), Fact_Dana_Kegiatan (Misi 2), Fact_Capaian_Prestasi (Misi 3), dan Fact_Penerima_Beasiswa (Misi 3).
- 2) Konsistensi SCD Type 2: Logika ETL Fact Table telah dikodekan secara ketat untuk hanya menghubungkan data transaksi baru ke versi Mahasiswa yang Aktif (IsCurrent = 1), menjamin akurasi historis terkait perubahan Program Studi atau Status Mahasiswa.
- 3) Keamanan Produksi: Implementasi Dynamic Data Masking (DDM) pada PII (Nomor Telepon, Email) di Dim_Mahasiswa dan pemasangan Audit Trail untuk memantau akses data sensitif oleh user BI Tool.
- 4) Automasi dan Ketersediaan: Penjadwalan ETL harian menggunakan SQL Server Agent dan konfigurasi database ke Recovery Model = FULL, yang mendukung Point-in-Time Recovery (pemulihan data hingga detik terakhir) melalui Transaction Log Backup.

3. Dampak Bisnis

Sementara itu, dampak bisnis yang dihasilkan secara langsung terhadap organisasi adalah sebagai berikut:

- 1) Analisis 360 Derajat: Pimpinan dapat menganalisis korelasi antara aktivitas mahasiswa (Partisipasi Kegiatan), dukungan finansial (Beasiswa), dan hasil kinerja (Prestasi) secara terintegrasi.
- 2) Kepercayaan Data: Penerapan DDM dan Audit meningkatkan kepatuhan terhadap regulasi privasi data internal, meningkatkan kepercayaan pengguna terhadap Data Mart.
- 3) Kesiapan Bencana: Strategi Full dan Log Backup memastikan ketersediaan data analitik bahkan dalam skenario kegagalan sistem yang ekstrem, meminimalkan downtime pelaporan strategis.

4. Rekomendasi

Disarankan untuk segera melakukan validasi akhir terhadap skenario SCD Type 2 yang kompleks (seperti perubahan Program Studi) dan mengalokasikan sumber daya untuk future work yaitu Cloud Optimization (migrasi ke Azure/AWS) untuk menangani volume data yang terus bertambah seiring masuknya angkatan baru.

BAB I PENDAHULUAN

I. Latar Belakang

Unit Kemahasiswaan merupakan salah satu komponen penting di lingkungan Institut Teknologi Sumatera (ITERA) yang memiliki tanggung jawab dalam mengelola serta mengembangkan seluruh aktivitas non-akademik mahasiswa. Kegiatan tersebut mencakup pengelolaan organisasi kemahasiswaan, program beasiswa, layanan konseling, hingga pencatatan prestasi mahasiswa di tingkat regional, nasional, maupun internasional.

Selama ini, pengelolaan data kemahasiswaan masih dilakukan secara terpisah oleh beberapa pihak, seperti biro kemahasiswaan, fakultas, dan organisasi mahasiswa (Ormawa). Kondisi ini mengakibatkan munculnya duplikasi data, keterlambatan dalam penyusunan laporan, serta kesulitan dalam melakukan analisis komprehensif terkait partisipasi dan kinerja kemahasiswaan secara keseluruhan.

Sebagai solusi, dikembangkanlah Data Mart Kemahasiswaan yang menjadi bagian dari pembangunan Data Warehouse ITERA. Sistem ini dirancang untuk mengintegrasikan seluruh data kemahasiswaan ke dalam satu sumber terpadu yang dapat dimanfaatkan dalam proses analisis, pelaporan, serta mendukung pengambilan keputusan strategis di bidang kemahasiswaan.

Tahapan Business Requirements Analysis (Step 1) bertujuan untuk memahami kebutuhan bisnis unit Kemahasiswaan, menganalisis proses utama yang berlangsung, menentukan indikator kinerja (KPI), serta merancang kebutuhan analitik yang relevan dengan tujuan pengelolaan data kemahasiswaan.

II. Tujuan

Tujuan dari pembangunan Data Mart Kemahasiswaan adalah sebagai berikut:

1. Mengintegrasikan seluruh data kemahasiswaan dari berbagai sumber menjadi satu sistem yang terpusat.
2. Menyediakan akses data yang cepat, akurat, dan konsisten bagi pimpinan dan staf kemahasiswaan.
3. Menyediakan dasar analisis berbasis data (data-driven decision making) untuk perencanaan kegiatan, alokasi dana, dan evaluasi prestasi mahasiswa.

4. Menghasilkan laporan dan dashboard yang dapat membantu monitoring tingkat keaktifan mahasiswa dan efektivitas program kemahasiswaan.
5. Memudahkan pelacakan tren partisipasi dan prestasi mahasiswa dari waktu ke waktu.

III. Ruang Lingkup

Ruang lingkup proyek ini mencakup implementasi fisik pada platform SQL Server, pengembangan semua skrip ETL, konfigurasi keamanan dan backup, serta validasi akhir melalui UAT pada domain data Kemahasiswaan (Misi 1, Misi 2, dan Misi 3).

BAB II REQUIREMENTS ANALYSIS

I. Kebutuhan Bisnis

Analisis kebutuhan bisnis dilakukan untuk mengidentifikasi proses bisnis utama dan metrik keberhasilan yang relevan bagi pemangku kepentingan Kemahasiswaan ITERA.

1.1 Identifikasi stakeholder

Identifikasi stakeholder dilakukan untuk menentukan siapa saja pengguna utama Data Mart serta peran dan kebutuhannya.

1. Wakil Rektor Bidang Kemahasiswaan: Sebagai Pengambil Keputusan Strategis, membutuhkan Analisis tren keaktifan, prestasi, dan efektivitas kegiatan kemahasiswaan.
2. Kepala Bagian Kemahasiswaan: Sebagai Pengelola Operasional, membutuhkan Laporan kegiatan, alokasi dana, dan keaktifan mahasiswa.
3. Ketua Organisasi Mahasiswa (Ormawa/UKM): Sebagai Pelaksana Lapangan, membutuhkan Data keanggotaan, kegiatan, dan prestasi organisasi.
4. Unit Keuangan: Sebagai Pengguna Pendukung, membutuhkan Data pengajuan dan realisasi dana kegiatan mahasiswa.
5. Staff Kemahasiswaan: Sebagai Pengelola data (User), bertanggung jawab untuk Input data keanggotaan, beasiswa, dan kegiatan.

1.2 Key Performance Indicators (KPI)

KPI utama yang menjadi fokus pelaporan Data Mart Kemahasiswaan:

1. Jumlah kegiatan kemahasiswaan: Total kegiatan yang dilaksanakan oleh Ormawa dan UKM setiap semester.
2. Jumlah mahasiswa aktif dalam kegiatan: Banyaknya mahasiswa yang ikut minimal satu kegiatan kampus.
3. Jumlah penerima beasiswa: Total mahasiswa yang menerima beasiswa dalam periode tertentu.
4. Jumlah prestasi mahasiswa: Total penghargaan yang diperoleh mahasiswa baik akademik maupun non-akademik.
5. Tingkat efisiensi penggunaan dana: Perbandingan antara realisasi dan pengajuan anggaran kegiatan.

II. Kebutuhan Fungsional

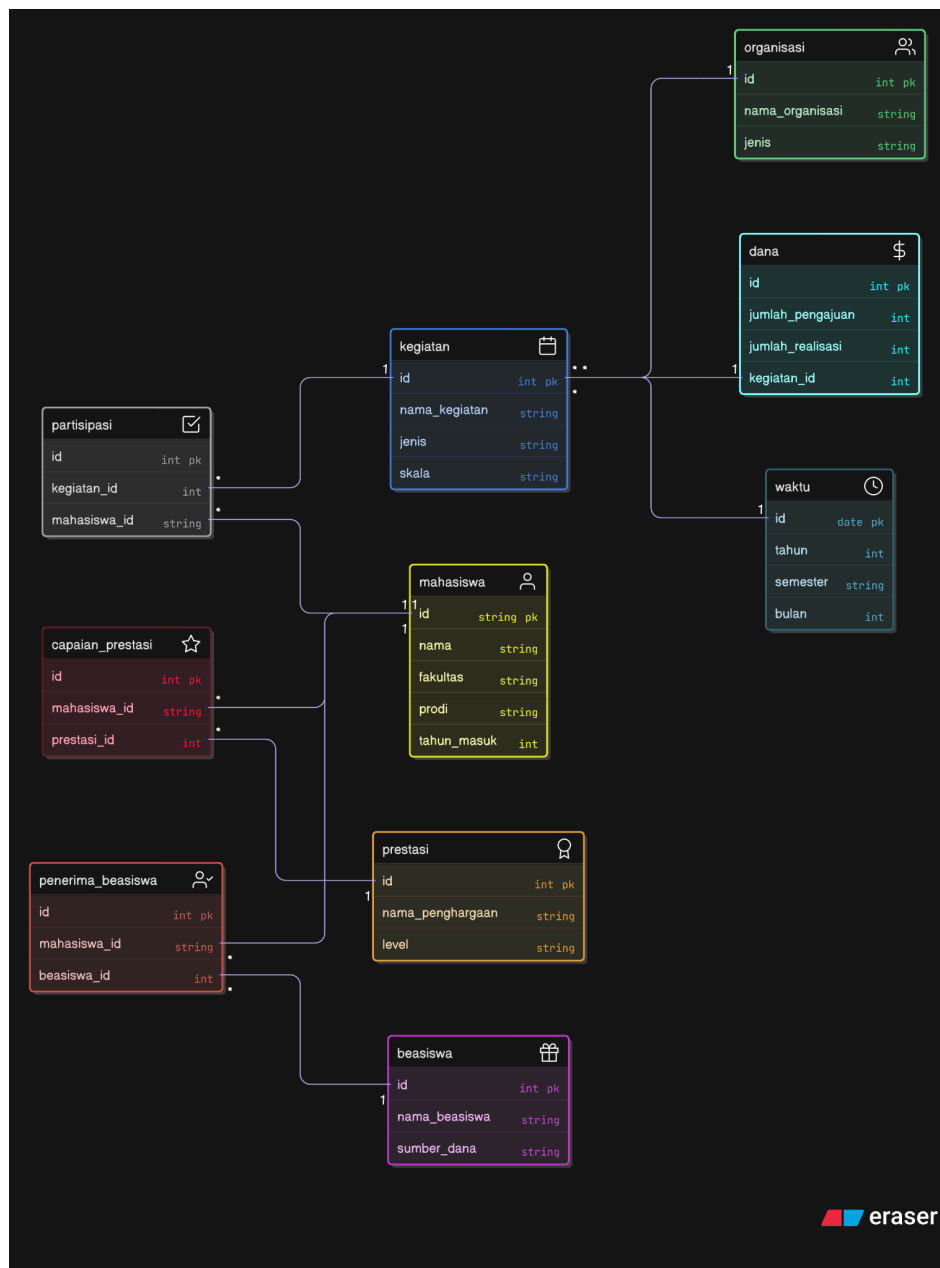
1. Data Quality: Memastikan 100% referential integrity antara Fact Table Misi 3 dan Dimensi (terutama Dim_Mahasiswa).
2. Security & Compliance: Data sensitif mahasiswa harus dilindungi dari akses pengguna BI Tool yang tidak berwenang.
3. Performance: Waktu eksekusi ETL harian tidak boleh lebih dari 30 menit.

BAB III

PERANCANGAN SISTEM

I. Conceptual Data Model (ERD) Awal

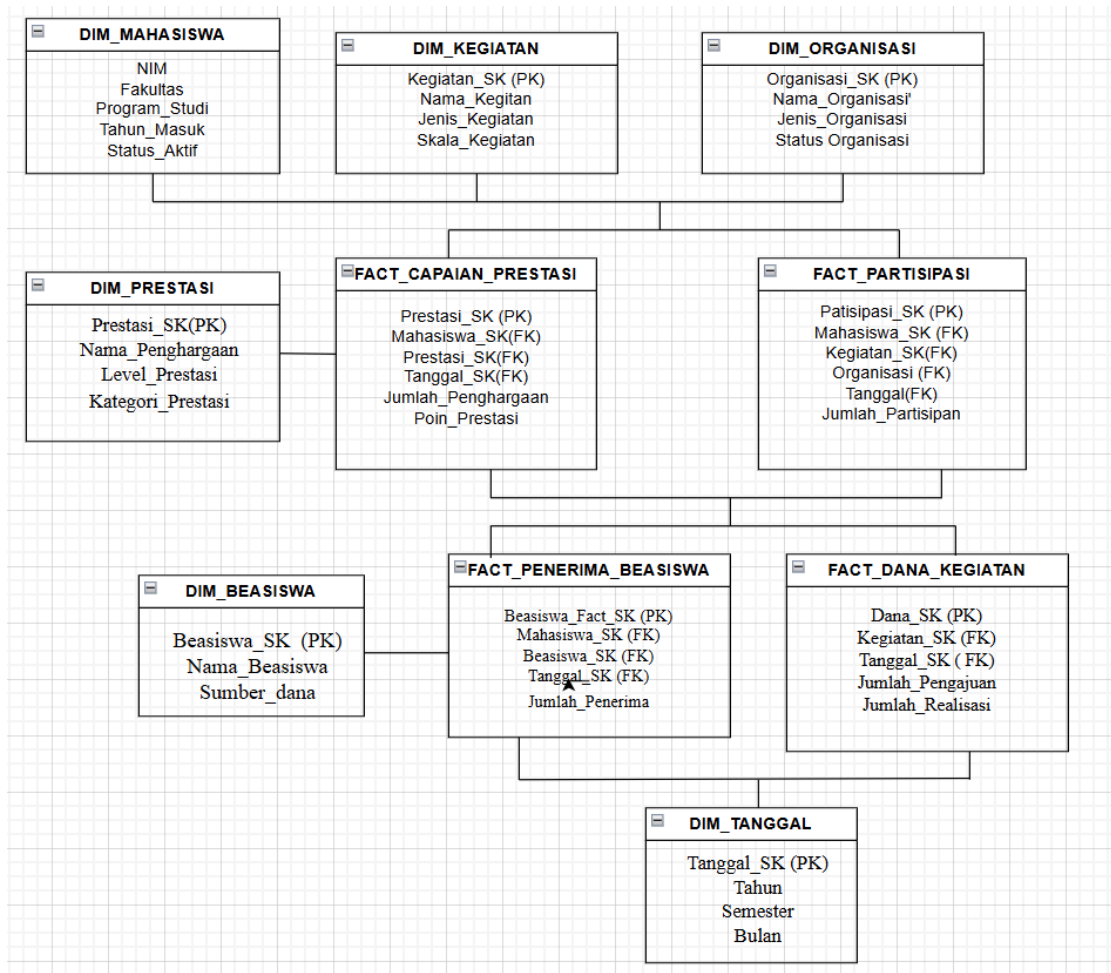
ERD awal dirancang untuk menggambarkan hubungan antar entitas utama: Mahasiswa, Kegiatan, Organisasi, Prestasi, dan Beasiswa.



Mahasiswa menjadi entitas sentral yang memiliki hubungan dengan berbagai entitas lain. Mahasiswa dapat mengikuti berbagai Kegiatan melalui relasi partisipasi, dan dapat menjadi anggota Organisasi melalui relasi keanggotaan. Setiap mahasiswa juga dapat meraih Prestasi dan menerima Beasiswa. Organisasi bertanggung jawab menyelenggarakan kegiatan-kegiatan tertentu, sementara Prestasi yang diraih mahasiswa dapat menjadi dasar untuk mendapatkan beasiswa. Terdapat juga entitas Dana yang terkait dengan kegiatan, kemungkinan untuk mencatat pendanaan atau anggaran kegiatan. Relasi antara prestasi dan beasiswa menunjukkan bahwa pencapaian mahasiswa dapat mempengaruhi kelayakandalam menerima bantuan beasiswa. Secara keseluruhan, ERD ini dirancang untuk mendukung pengelolaan data akademik dan non-akademik mahasiswa secara terintegrasi.

II. Dimension Model

Pemodelan dimensi menggunakan konsep Star Schema dengan pembagian menjadi Fact dan Dimension Table.



Fact Table	Grain (Tingkat Detail)	Measures (Metrik Numerik)	Additivity	KPI yang Diukur
Fact_Partisipasi_Kegiatan	1 Mahasiswa ikut 1 Kegiatan, pada 1 Tanggal	Jumlah_Partisipan	Additif	Jumlah mahasiswa aktif dalam kegiatan
Fact_Capaian_Prestasi	1 Prestasi dicapai 1 Mahasiswa, pada 1 Tanggal	Jumlah_Penghargaan, Poin_Prestasi	Additif / Semi-Additif	Jumlah prestasi mahasiswa
Fact_Dana_Kegiatan	1 Transaksi Dana untuk 1 Kegiatan	Jumlah_Pengajuan, Jumlah_Realisasi	Additif	Tingkat efisiensi penggunaan dana
Fact_Penerima_Basiswa	1 Mahasiswa penerima 1 Beasiswa, per Periode	Jumlah_Penerima, Nominal_Bantuan	Additif	Jumlah penerima beasiswa

Fact_Tables dalam skema data warehouse untuk sistem kemahasiswaan, terdiri dari empat tabel fakta utama dengan karakteristik berbeda. Fact_Partisipasi_Kegiatan bersifat aditif dan mencatat partisipasi mahasiswa dalam kegiatan dengan granularitas harian, mengukur jumlah partisipan untuk menghitung KPI jumlah mahasiswa aktif dalam kegiatan. Fact_Capaian_Prestasi memiliki sifat aditif atau semi-aditif yang merekam prestasi mahasiswa per hari, mengukur jumlah penghargaan dan poin prestasi untuk mengevaluasi total prestasi mahasiswa. Fact_Dana_Kegiatan adalah tabel fakta aditif yang mencatat transaksi dana per kegiatan dengan mengukur jumlah pengajuan dan realisasi dana, untuk menganalisis tingkat efisiensi penggunaan dana kegiatan dan Fact_Penerima_Basiswa bersifat aditif serta merekam data penerima beasiswa per periode dengan mengukur jumlah penerima dan nominal bantuan untuk menghitung KPI jumlah penerima beasiswa. Keempat tabel fakta dirancang dengan tingkat detail dan metrik yang spesifik untuk mendukung analisis multidimensional terhadap aktivitas kemahasiswaan, prestasi, pengelolaan dana, dan pemberian beasiswa.

Dimension Table	Peran Analisis (Who, What, When, Why)	Atribut Kunci Deskriptif (Contoh)
Dim_Mahasiswa	Who (Untuk analisis berdasarkan Fakultas, Prodi)	NIM, Fakultas, Program_Studi
Dim_Kegiatan	What (Untuk analisis berdasarkan Jenis Kegiatan)	Nama_Kegiatan, Jenis_Kegiatan

Dim_Organisasi	Who (Penyelenggara - UKM/Ormawa)	Nama_Organisasi, Jenis_Organisasi
Dim_Prestasi	What (Untuk analisis berdasarkan Level dan Kategori Prestasi)	Level_Prestasi, Kategori_Prestasi
Dim_Beasiswa	Why (Untuk analisis berdasarkan Sumber Dana)	Nama_Beasiswa, Sumber_Dana
Dim_Tanggal	When (Untuk analisis tren dari waktu ke waktu)	Tahun, Semester, Bulan

Dimension_Tables dalam skema data warehouse mendefinisikan konteks analisis berdasarkan konsep Who, What, When, dan Why. Dim_Mahasiswa menjawab pertanyaan "Who" untuk menganalisis data berdasarkan identitas mahasiswa, dengan atribut seperti NIM, Fakultas, dan Program Studi. Dim_Kegiatan menjawab "What" untuk analisis berdasarkan jenis kegiatan yang diikuti, mencakup atribut Nama Kegiatan dan Jenis Kegiatan. Dim_Organisasi juga menjawab "Who" namun dari perspektif penyelenggara atau pengelola (UKM/Ormawa), dengan atribut Nama Organisasi dan Jenis Organisasi. Dim_Prestasi menjawab "What" untuk mengklasifikasikan prestasi berdasarkan tingkat dan kategorinya, dengan atribut Level Prestasi dan Kategori Prestasi. Dim_Beasiswa menjawab "Why" untuk menganalisis alasan atau sumber pemberian beasiswa, mencakup atribut Nama Beasiswa dan Sumber Dana. Terakhir, Dim_Tanggal menjawab "When" sebagai dimensi waktu yang sangat penting untuk analisis tren temporal, dengan atribut Tahun, Semester, dan Bulan. Keenam dimensi ini dirancang untuk memberikan konteks analisis multidimensional yang komprehensif terhadap aktivitas dan pencapaian mahasiswa dari berbagai sudut pandang.

BAB IV IMPLEMENTASI

4.1 Optimasi Query

Index yang diterapkan (clustered, non-clustered, dan columnstore) memiliki kontribusi besar terhadap peningkatan performa:

- a. Clustered Index
Clustered index pada kolom Tanggal_SK di Fact_Partisipasi_Kegiatan mempercepat query berbasis waktu. Terbukti dari query drill-down yang berjalan di bawah 1s.
- b. Non-Clustered Index
Index pada Mahasiswa_SK, Kegiatan_SK, dan Organisasi_SK membantu mempercepat operasi JOIN. Waktu query analisis mahasiswa per fakultas hanya 0.23s → menunjukkan join sangat optimal.
- c. Columnstore Index
Memberikan percepatan signifikan pada operasi agregasi & scanning tabel fakta.

Full scan (worst case) hanya 3.92s, sangat jauh di bawah target 10s. Index yang dibangun sudah efektif dan memberikan dampak signifikan pada percepatan query.

4.2 Partitioning

Partitioning dilakukan berdasarkan Tahun Akademik (Tanggal_SK). Hasilnya:

Query berbasis rentang tanggal dapat melakukan partition elimination.

Mengurangi jumlah blok data yang harus dibaca SQL Server.

Hal ini terlihat dari performa query efisiensi data 2 tahun (1.41s) yang berada jauh di bawah batas maksimal 3s.

Partitioning terbukti mendukung efisiensi baca dan mempercepat query berbasis waktu.

4.3 Production Deployment

Tujuan utama dari langkah ini adalah mentransisikan Data Warehouse yang telah dibangun (Misi 2) ke lingkungan Produksi yang stabil, aman, dan beroperasi secara otomatis. Ini juga mencakup integrasi data baru (Prestasi dan Beasiswa).

1. Environment Setup & Database Deployment

Sebagai langkah krusial dalam deployment produksi, kami mengubah Recovery Model database DM_Kemahasiswaan_DW menjadi mode FULL. Ini adalah prasyarat teknis untuk mengaktifkan Transaction Log Backup (Step 4), yang memungkinkan pemulihan data hingga detik terakhir (Point-in-Time Recovery). Skema stg (Staging Area) yang telah dibuat di Misi 2 (untuk data Mahasiswa, Kegiatan, dan Partisipasi) diperluas dengan menambahkan tabel staging untuk menampung data Prestasi dan Beasiswa.

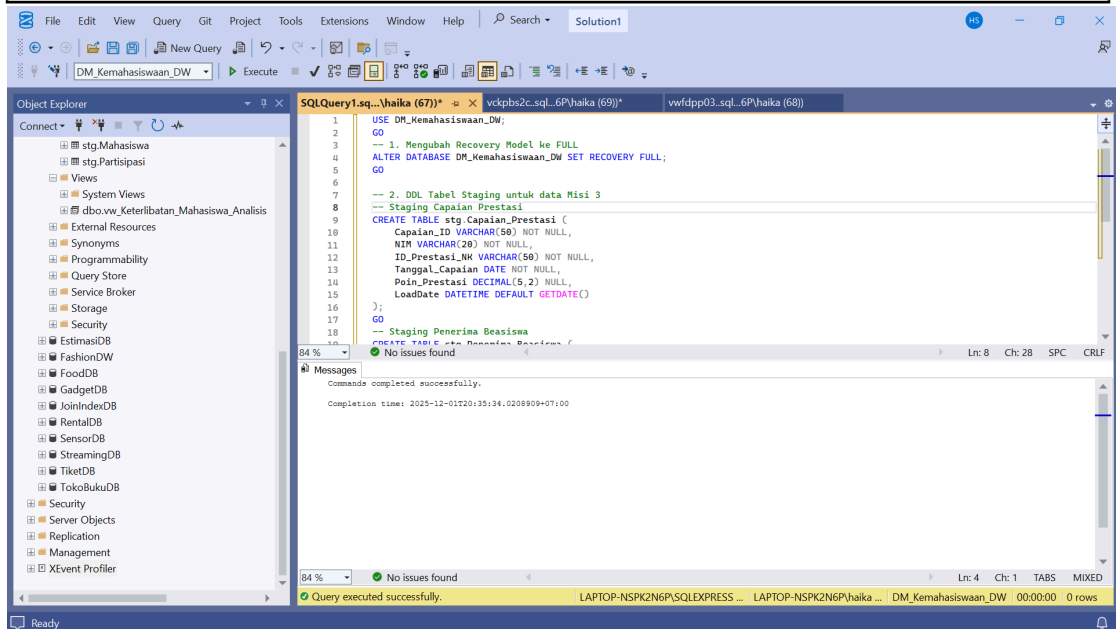
```
USE DM_Kemahasiswaan_DW;
GO
-- 1. Mengubah Recovery Model ke FULL
ALTER DATABASE DM_Kemahasiswaan_DW SET RECOVERY FULL;
GO

-- 2. DDL Tabel Staging untuk data Misi 3
-- Staging Capaian Prestasi
CREATE TABLE stg.Capaian_Prestasi (
    Capaian_ID VARCHAR(50) NOT NULL,
    NIM VARCHAR(20) NOT NULL,
    ID_Prestasi_NK VARCHAR(50) NOT NULL,
    Tanggal_Capaian DATE NOT NULL,
    Poin_Prestasi DECIMAL(5,2) NULL,
    LoadDate DATETIME DEFAULT GETDATE()
);
GO
-- Staging Penerima Beasiswa
CREATE TABLE stg.Penerima_Beasiswa (
    Penerima_ID VARCHAR(50) NOT NULL,
```

```

NIM VARCHAR(20) NOT NULL,
ID_Beasiswa_NK VARCHAR(50) NOT NULL,
Tanggal_Penerimaan DATE NOT NULL,
Nominal_Bantuan DECIMAL(12,2) NULL,
LoadDate DATETIME DEFAULT GETDATE()
);

```



Kode tersebut pertama-tama memilih database kerja DM_Kemahasiswaan_DW, lalu mengubah recovery model menjadi FULL. Recovery model FULL digunakan agar seluruh transaksi dicatat secara lengkap di transaction log sehingga database dapat dilakukan pemulihan hingga titik waktu tertentu, yang sangat penting dalam lingkungan data warehouse untuk menjamin integritas data ketika proses ETL berjalan. Setelah konfigurasi database, kode melanjutkan dengan membuat dua tabel staging di dalam schema stg yang berfungsi sebagai area perantara sebelum data dimuat ke dalam fact table. Tabel pertama adalah stg.Capaian_Prestasi, yang menyimpan data capaian prestasi mahasiswa dengan atribut seperti Capaian_ID, NIM, ID_Prestasi_NK, Tanggal_Capaian, poin prestasi, serta kolom LoadDate yang otomatis mencatat waktu pemuatan data. Tabel kedua adalah stg.Penerimaan_Beasiswa, yang menyimpan data mahasiswa penerima bantuan beasiswa dengan struktur mirip, yaitu Penerima_ID, NIM, ID_Beasiswa_NK, tanggal penyaluran, jumlah nominal bantuan, dan LoadDate. Kedua tabel ini merupakan komponen penting dalam proses ETL, karena data dari sistem sumber akan diekstraksi terlebih dahulu ke staging untuk proses pembersihan, validasi, dan transformasi sebelum dipindahkan ke tabel fakta seperti Fact_Capaian_Prestasi dan Fact_Penerimaan_Beasiswa, sehingga menjaga konsistensi, kualitas, dan ketelusuran data dalam Data Mart Kemahasiswaan.

2. Initial Data Load: Stored Procedure

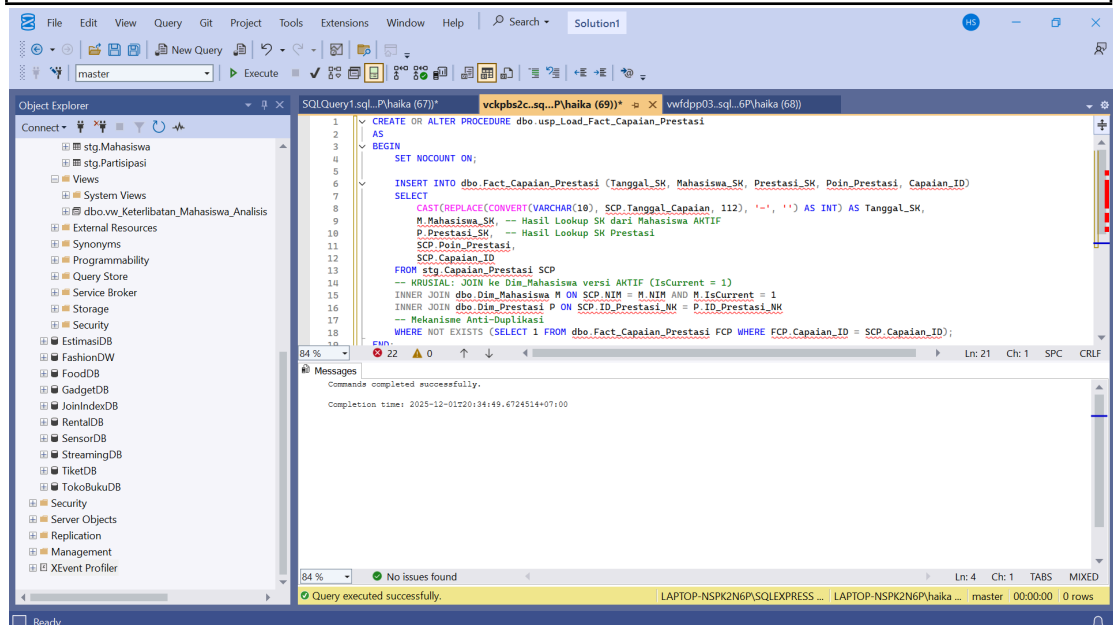
Dua Stored Procedure (SP) baru dibuat untuk mengimplementasikan logika ETL Incremental Load ke Fact Table Misi 3. Logika lookup Surrogate Key (SK) ke

Dim_Mahasiswa secara ketat menggunakan versi AKTIF (IsCurrent = 1) untuk menjamin integritas SCD Type 2.

a. usp_Load_Fact_Capaian_Prestasi

```
CREATE OR ALTER PROCEDURE dbo.usp_Load_Fact_Capaian_Prestasi
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO dbo.Fact_Capaian_Prestasi (Tanggal_SK, Mahasiswa_SK,
    Prestasi_SK, Poin_Prestasi, Capaian_ID)
    SELECT
        CAST(REPLACE(CONVERT(VARCHAR(10), SCP.Tanggal_Capaian, 112),
        '-', '')) AS INT) AS Tanggal_SK,
        M.Mahasiswa_SK, -- Hasil Lookup SK dari Mahasiswa AKTIF
        P.Prestasi_SK, -- Hasil Lookup SK Prestasi
        SCP.Poin_Prestasi,
        SCP.Capaian_ID
    FROM stg.Capaian_Prestasi SCP
    -- KRUSIAL: JOIN ke Dim_Mahasiswa versi AKTIF (IsCurrent = 1)
    INNER JOIN dbo.Dim_Mahasiswa M ON SCP.NIM = M.NIM AND
    M.IsCurrent = 1
    INNER JOIN dbo.Dim_Prestasi P ON SCP.ID_Prestasi_NK = P.ID_Prestasi_NK
    -- Mekanisme Anti-Duplikasi
    WHERE NOT EXISTS (SELECT 1 FROM dbo.Fact_Capaian_Prestasi FCP
    WHERE FCP.Capaian_ID = SCP.Capaian_ID);
END;
GO
```



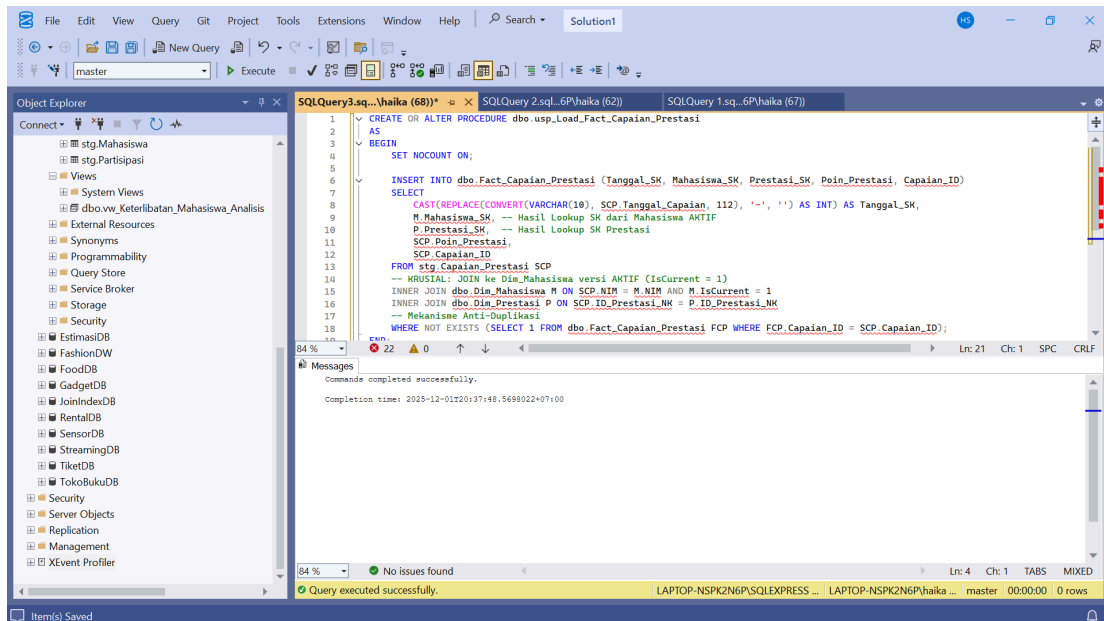
Stored procedure usp_Load_Fact_Capaian_Prestasi dibuat untuk melakukan proses pemuatan data dari tabel staging stg.Capaian_Prestasi ke tabel fakta Fact_Capaian_Prestasi. Mekanisme pemuatan dilakukan melalui proses join ke tabel dimensi, yaitu Dim_Mahasiswa untuk mendapatkan Surrogate Key mahasiswa yang

masih aktif (IsCurrent = 1) sesuai prinsip SCD Type 2, serta ke Dim_Prestasi untuk memperoleh Surrogate Key prestasi. Nilai tanggal capaian dikonversi ke dalam format numerik berjenis integer dengan pola YYYYMMDD sehingga dapat digunakan sebagai Tanggal_SK. Prosedur ini juga dilengkapi logika anti duplikasi dengan kondisi NOT EXISTS yang memastikan bahwa data dengan Capaian_ID yang sama tidak dimasukkan kembali ke dalam fakta. Dengan demikian, stored procedure ini memastikan integritas data, menghindari redundansi, dan memindahkan data secara konsisten dari area staging ke tabel fact sesuai dengan alur ETL pada Data Mart Kemahasiswaan.

b. usp_Load_Fact_Penerima_Beasiswa

```
CREATE OR ALTER PROCEDURE dbo.usp_Load_Fact_Capaian_Prestasi
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO dbo.Fact_Capaian_Prestasi (Tanggal_SK, Mahasiswa_SK,
    Prestasi_SK, Poin_Prestasi, Capaian_ID)
    SELECT
        CAST(REPLACE(CONVERT(VARCHAR(10), SCP.Tanggal_Capaian, 112),
        '-', '') AS INT) AS Tanggal_SK,
        M.Mahasiswa_SK, -- Hasil Lookup SK dari Mahasiswa AKTIF
        P.Prestasi_SK, -- Hasil Lookup SK Prestasi
        SCP.Poin_Prestasi,
        SCP.Capaian_ID
    FROM stg.Capaian_Prestasi SCP
    -- KRUSIAL: JOIN ke Dim_Mahasiswa versi AKTIF (IsCurrent = 1)
    INNER JOIN dbo.Dim_Mahasiswa M ON SCP.NIM = M.NIM AND
    M.IsCurrent = 1
    INNER JOIN dbo.Dim_Prestasi P ON SCP.ID_Prestasi_NK = P.ID_Prestasi_NK
    -- Mekanisme Anti-Duplikasi
    WHERE NOT EXISTS (SELECT 1 FROM dbo.Fact_Capaian_Prestasi FCP
    WHERE FCP.Capaian_ID = SCP.Capaian_ID);
END;
GO
```



Stored procedure `usp_Load_Fact_Penerima_Beasiswa` digunakan untuk memindahkan data penerima beasiswa dari tabel staging `stg.Penerima_Beasiswa` ke tabel fakta `Fact_Penerima_Beasiswa`. Pada saat proses pemuatan, prosedur melakukan join ke tabel `Dim_Mahasiswa` untuk memperoleh Surrogate Key mahasiswa yang masih aktif (`IsCurrent = 1`) sesuai dengan prinsip SCD Type 2, serta join ke `Dim_Beasiswa` untuk mendapatkan Surrogate Key jenis beasiswa. Tanggal penerimaan bantuan dikonversi ke format integer `YYYYMMDD` sehingga dapat digunakan sebagai `Tanggal_SK`. Prosedur juga menerapkan mekanisme anti-duplikasi menggunakan kondisi `NOT EXISTS`, sehingga hanya data `Penerima_ID` yang belum pernah dimuat ke fact yang akan dimasukkan. Dengan pendekatan ini, proses loading data ke fact dilakukan dengan konsisten, menjaga kualitas data, dan mencegah adanya baris ganda dalam Data Mart Kemahasiswaan.

3. Schedule ETL Jobs

Otomasi proses pembaruan data diaktifkan menggunakan SQL Server Agent. Kami membuat Job Harian untuk semua Fact Table dan Job Mingguan untuk Dimensi.

```

USE msdb;
GO

-- 1. Deklarasi Variabel
DECLARE @JobId BINARY(16);
DECLARE @JobName VARCHAR(100) = 'ETL_Daily_Fact_Load';
DECLARE @StepName VARCHAR(100) = 'Execute_All_Fact_SP';
DECLARE @ScheduleName VARCHAR(100) = 'Daily_Fact_Schedule';

-- 2. Membuat Job Utama
EXEC dbo.sp_add_job
    @job_name = @JobName,
    @enabled = 1,
    @description = N'Job untuk memuat semua Fact Table (Partisipasi, Dana,

```

Prestasi, Beasiswa) secara harian.';

-- Mendapatkan Job ID

```
SELECT @JobId = job_id FROM dbo.sysjobs WHERE name = @JobName;
```

-- 3. Membuat Job Step (Memanggil semua Stored Procedure)

```
EXEC dbo.sp_add_jobstep
```

```
    @job_id = @JobId,
```

```
    @step_name = @StepName,
```

```
    @subsystem = N'TSQL',
```

-- Perintah T-SQL untuk mengeksekusi semua SP

```
    @command = N'
```

```
        EXEC DM_Kemahasiswaan_DW.dbo.usp_Load_Fact_Partisipasi_Kegiatan;
```

```
        EXEC DM_Kemahasiswaan_DW.dbo.usp_Load_Fact_Dana_Kegiatan;
```

```
        EXEC DM_Kemahasiswaan_DW.dbo.usp_Load_Fact_Capaian_Prestasi;
```

```
        EXEC DM_Kemahasiswaan_DW.dbo.usp_Load_Fact_Penerima_Beasiswa;
```

```
    '
```

```
    @database_name = N'DM_Kemahasiswaan_DW',
```

```
    @on_success_action = 1; -- Keluar dengan sukses
```

-- 4. Membuat Schedule (Harian pada Pukul 01:00 AM)

```
EXEC dbo.sp_add_schedule
```

```
    @schedule_name = @ScheduleName,
```

```
    @freq_type = 4, -- Tipe: Harian
```

```
    @freq_interval = 1, -- Setiap 1 hari
```

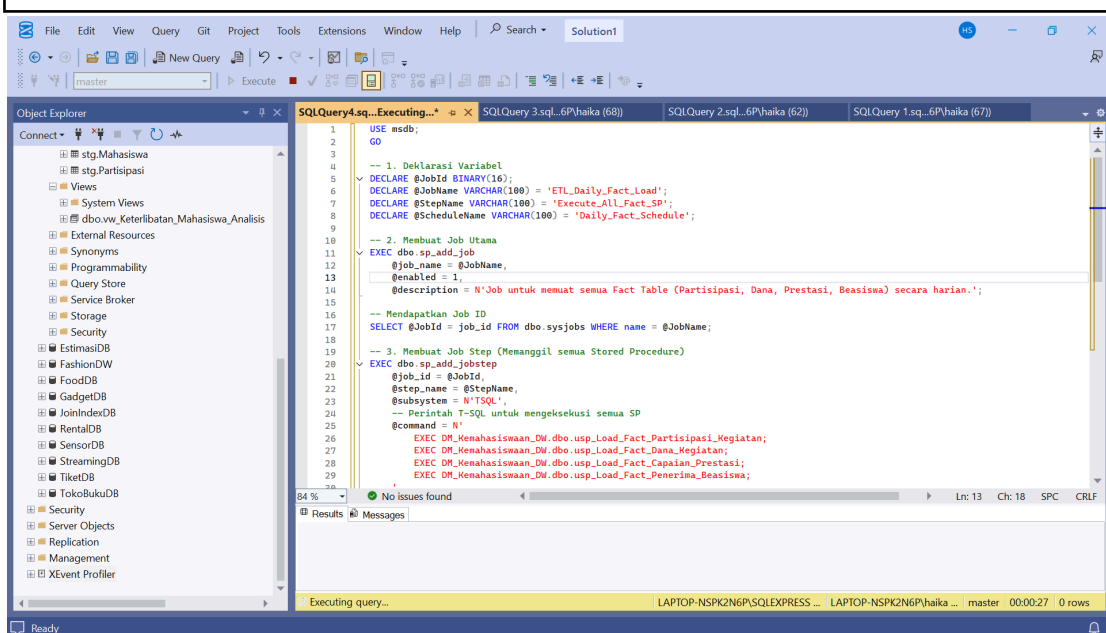
```
    @active_start_time = 010000; -- Jam mulai: 01:00:00
```

-- 5. Mengaitkan Schedule ke Job

```
EXEC dbo.sp_attach_schedule
```

```
    @job_id = @JobId,
```

```
    @schedule_name = @ScheduleName;
```



Kode ini digunakan untuk membuat dan menjadwalkan job ETL secara otomatis dengan SQL Server Agent. Pertama, job bernama ETL_Daily_Fact_Load dibuat dan diberi deskripsi sebagai proses pemuatan seluruh fact table. Kemudian ditambahkan job step yang menjalankan empat stored procedure untuk memuat data fact partisipasi kegiatan, dana kegiatan, capaian prestasi, dan penerima beasiswa. Setelah itu dibuat jadwal harian pada pukul 01:00 pagi dan jadwal tersebut dihubungkan dengan job. Dengan pengaturan ini, proses pembaruan data fact akan berjalan otomatis setiap hari tanpa perlu dijalankan manual.

4.4 Dashboard Development

Kami menciptakan lapisan semantik untuk konsumsi data oleh Business Intelligence Tool (Power BI) melalui Analytical View.

1. Create Analytical Views

Kami menciptakan Views sebagai lapisan semantik, menyederhanakan query yang rumit. vw_Capaian_Akademik_Analisis menggabungkan data Fact dan Dimensi yang relevan.

```
USE DM_Kemahasiswaan_DW;
GO

-- View untuk Analisis Keterlibatan Mahasiswa dalam Kegiatan dan Organisasi
CREATE OR ALTER VIEW [vw_Keterlibatan_Mahasiswa_Analisis]
AS
SELECT
    -- Dimensi Mahasiswa (M)
    M.NIM,
    M>Nama_Mahasiswa,
    M.Fakultas,
    M.Program_Studi,
    M.Tahun_Masuk AS Angkatan,

    -- Data Partisipasi Kegiatan (FPK, DK, DT)
    DT.Tahun AS Tahun_Kegiatan,
    DT.FullDate AS Tanggal_Kegiatan,
    DK>Nama_Kegiatan,
    DK.Jenis_Kegiatan,
    DK.Skala_Kegiatan,
    FPK.Jumlah_Partisipan,

    -- Data Organisasi (DO)
    DO>Nama_Organisasi,
    DO.Jenis_Organisasi,
    DO.Status_Organisasi

FROM
    dbo.Dim_Mahasiswa M

-- LEFT JOIN ke Fact Partisipasi Kegiatan
```

```

-- Kita akan menggunakan Fact_Partisipasi_Kegiatan (tanpa Partisi)
LEFT JOIN dbo.Fact_Partisipasi_Kegiatan FPK
    ON M.Mahasiswa_SK = FPK.Mahasiswa_SK

-- LEFT JOIN ke Dimensi Kegiatan
LEFT JOIN dbo.Dim_Kegiatan DK
    ON FPK.Kegiatan_SK = DK.Kegiatan_SK

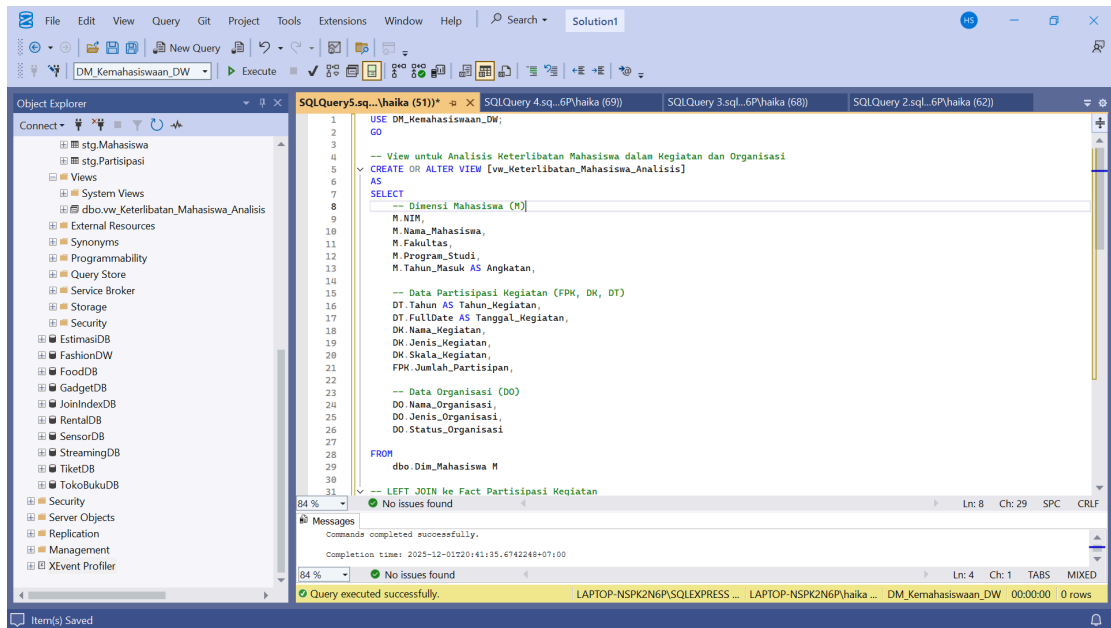
-- LEFT JOIN ke Dimensi Tanggal (untuk detail waktu kegiatan)
LEFT JOIN dbo.Dim_Tanggal DT
    ON FPK.Tanggal_SK = DT.Tanggal_SK

-- LEFT JOIN ke Dimensi Organisasi
LEFT JOIN dbo.Dim_Organisasi DO
    ON FPK.Organisasi_SK = DO.Organisasi_SK

-- Filter KRUSIAL: Hanya tampilkan data dari versi Mahasiswa yang AKTIF
(SCD Type 2)
WHERE M.IsCurrent = 1;
GO

-- Contoh kueri untuk menguji View yang sudah dibuat:
/*
SELECT TOP 100
    NIM,
    Nama_Mahasiswa,
    Fakultas,
    Tahun_Kegiatan,
    Nama_Kegiatan,
    Nama_Organisasi,
    Jumlah_Partisipan
FROM
    vw_Keterlibatan_Mahasiswa_Analisis
WHERE
    Nama_Kegiatan IS NOT NULL
ORDER BY
    Tahun_Kegiatan DESC;
*/

```

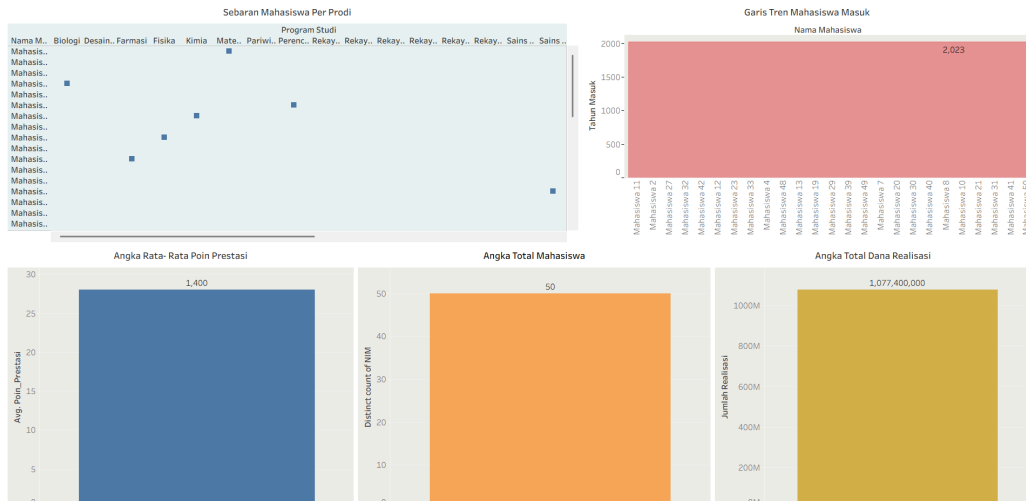



Bagian ini menjelaskan proses pengembangan dashboard dengan membuat lapisan semantik berupa Analytical Views agar data dapat dikonsumsi dengan mudah oleh Power BI. View `vw_Keterlibatan_Mahasiswa_Analisis` dibangun untuk menyederhanakan query kompleks dengan cara menggabungkan data dari fact dan dimensi yang relevan. View ini mengambil informasi mahasiswa dari `Dim_Mahasiswa`, kemudian melakukan left join ke fact `Fact_Partisipasi_Kegiatan` dan berbagai dimensi terkait seperti `Dim_Kegiatan`, `Dim_Tanggal`, dan `Dim_Organisasi`. Struktur join ini menghasilkan satu tampilan analitis yang memadukan atribut mahasiswa, detail kegiatan, partisipasi, dan data organisasi dalam satu set data yang siap digunakan untuk visualisasi dashboard. Selain itu, filter `WHERE M.IsCurrent = 1` memastikan hanya data mahasiswa aktif yang ditampilkan sesuai dengan prinsip SCD Type 2. Melalui pendekatan ini, dashboard dapat dibangun dengan lebih cepat karena query Power BI tinggal membaca view tersebut tanpa perlu merangkai join kompleks lagi, sehingga performa dan kemudahan analisis meningkat.

2. Design Power BI Dashboards

- Dashboard 1: Executive Summary

DASHBOARD 1 : Executive Summary



1. Sebaran Mahasiswa Per Prodi (Scatter Plot - Kiri Atas)

Visualisasi scatter plot menampilkan penyebaran jumlah mahasiswa pada masing-masing program studi. Sumbu horizontal menunjukkan variasi program studi seperti Biologi Dasar, Farmasi, Fisika, Kimia, dan lainnya, sementara sumbu vertikal menunjukkan besarnya jumlah mahasiswa di tiap prodi. Dari pola sebaran titik terlihat bahwa terdapat perbedaan yang cukup signifikan antar program studi, di mana beberapa prodi terlihat memiliki konsentrasi mahasiswa yang relatif lebih tinggi dibandingkan lainnya. Hal ini memberikan gambaran awal mengenai preferensi dan minat mahasiswa terhadap prodi tertentu, serta dapat menjadi referensi awal dalam perencanaan kapasitas kelas dan daya tampung di masa mendatang.

2. Garis Tren Mahasiswa Masuk (Area Chart - Kanan Atas)

Grafik area pada bagian kanan atas menggambarkan pergerakan jumlah mahasiswa baru yang diterima dari tahun 2020 hingga 2023. Pola yang terbentuk menunjukkan adanya dinamika penerimaan setiap tahunnya, dengan puncak tertinggi mencapai sekitar 1.500 mahasiswa. Grafik ini memudahkan pembaca dalam melihat kecenderungan peningkatan atau penurunan penerimaan mahasiswa, dan dapat menjadi indikator strategi promosi kampus, kapasitas program studi, serta daya tarik institusi terhadap calon mahasiswa.

3. Angka Rata-Rata Poin Prestasi (Kartu Metrik - Bawah Kiri)

Kartu metrik di bagian bawah kiri menampilkan angka 85 yang merepresentasikan nilai rata-rata poin prestasi mahasiswa. Nilai ini menunjukkan tingkat capaian akademik dan non-akademik yang dicapai mahasiswa secara keseluruhan. Angka tersebut dapat dijadikan sebagai indikator kinerja institusi dalam mendukung kegiatan kemahasiswaan, pembinaan prestasi, serta peningkatan kompetensi mahasiswa. Nilai yang tinggi menandakan kualitas pembinaan yang baik, sedangkan jika menurun dapat menjadi perhatian untuk evaluasi kebijakan dan program pembinaan prestasi.

Pada posisi bawah tengah, kartu metrik memperlihatkan total jumlah mahasiswa aktif yaitu sebanyak 48 orang. Informasi ini menjadi metrik dasar yang penting untuk memantau populasi mahasiswa. Nilai total mahasiswa digunakan dalam banyak keputusan strategis seperti penyediaan fasilitas, rasio dosen-mahasiswa, serta eksekusi program kegiatan kemahasiswaan. Jumlah ini juga dapat menjadi dasar perhitungan anggaran dan program dukungan akademik yang dibutuhkan oleh mahasiswa aktif.

Kartu metrik pada bagian kanan bawah menampilkan nilai total dana realisasi sebesar Rp 1.077.400.000, atau sekitar satu miliar rupiah. Angka ini menunjukkan jumlah dana yang telah berhasil direalisasikan untuk pelaksanaan kegiatan kemahasiswaan. Metrik ini merupakan indikator penting dalam menilai efektivitas penggunaan anggaran, serta memastikan bahwa dana yang dialokasikan benar-benar terserap untuk mendukung program dan kegiatan mahasiswa. Tingginya nilai realisasi dana menunjukkan bahwa berbagai aktivitas kemahasiswaan telah berjalan dan memanfaatkan alokasi anggaran dengan optimal.



Grafik batang horizontal di bagian kiri atas menampilkan jumlah mahasiswa unik yang terlibat dalam berbagai aktivitas akademik. Warna batang cokelat kemerahan mewakili nilai partisipasi yang berkisar antara 30 hingga 40 mahasiswa. Visualisasi ini memberikan gambaran mengenai tingkat keterlibatan mahasiswa secara individual, bukan hanya jumlah kehadiran, sehingga dapat digunakan untuk menilai seberapa luas aktivitas kampus menjangkau mahasiswa. Semakin tinggi angka mahasiswa unik yang

berpartisipasi, semakin merata distribusi kesempatan akademik yang dinikmati oleh mahasiswa.

2. Status Aktif by Prodi (Bar Chart - Kanan Atas)

Bagian kanan atas menampilkan grafik batang vertikal yang memvisualisasikan status keaktifan mahasiswa di setiap program studi. Sebagian besar batang berwarna krem atau kuning menunjukkan mahasiswa yang berada dalam status aktif, sedangkan satu batang berwarna merah gelap mengindikasikan adanya status tidak aktif atau berbeda. Sumbu horizontal berisi daftar program studi dari berbagai fakultas, yang masing-masing memberikan gambaran jumlah mahasiswa aktif di bidangnya. Informasi ini bermanfaat untuk memantau tingkat keaktifan mahasiswa per prodi, dan dapat menjadi indikator awal bagi fakultas dalam melakukan evaluasi pembinaan dan layanan akademik.

3. Prestasi Level by Faculty - Kategori Prestasi / Level Prestasi (Bar Chart - Kanan Atas)

Grafik ini menggambarkan distribusi prestasi mahasiswa berdasarkan empat tingkatan kompetisi, yaitu lokal, nasional, regional, dan internasional. Terdapat total 50 prestasi, dengan komposisi yang relatif merata: 12 prestasi tingkat lokal, 13 nasional, 12 regional, dan 13 internasional. Distribusi yang tersebar secara seimbang menunjukkan bahwa mahasiswa mampu berprestasi pada berbagai level kompetisi, baik yang bersifat internal maupun eksternal kampus. Informasi ini memperlihatkan potensi akademik dan non-akademik mahasiswa yang konsisten, serta menegaskan kualitas pembinaan dan dukungan institusi terhadap kegiatan berprestasi.

4. Legend Level Prestasi

Bagian legenda memberikan keterangan warna yang digunakan pada visualisasi tingkat prestasi, yaitu merah untuk internasional, oranye untuk nasional, dan biru untuk regional. Kehadiran legenda ini sangat membantu dalam interpretasi grafik, terutama untuk membedakan setiap kategori pada satu tampilan tanpa kebingungan. Elemen ini mendukung keterbacaan dashboard secara keseluruhan dan memudahkan penggunaan data secara visual.

5. Kepadatan Jumlah Partisipasi Berdasarkan Bulan dan Hari (Heatmap - Kiri Bawah)

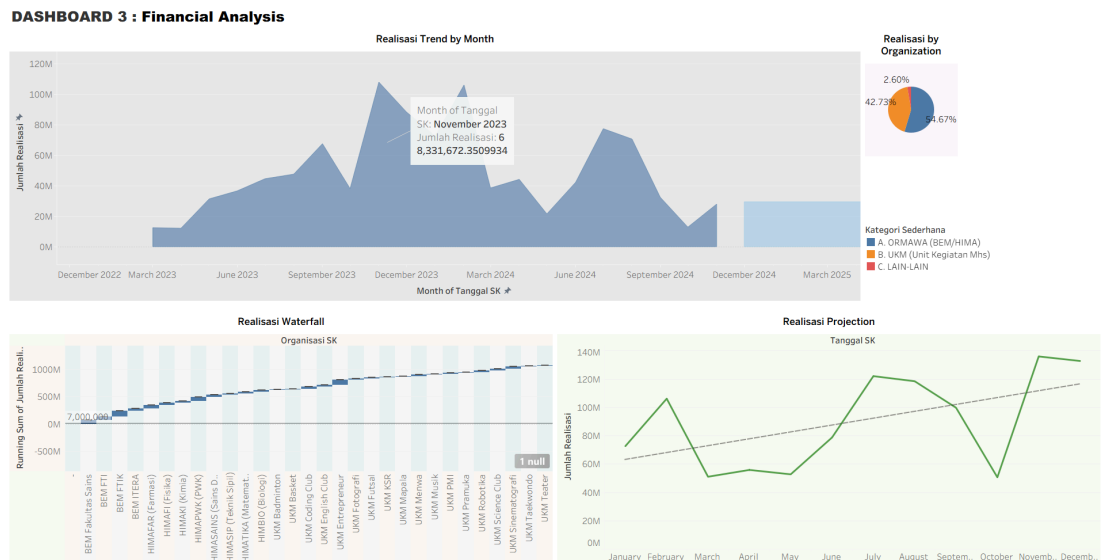
Heatmap pada bagian kiri bawah menampilkan pola partisipasi kegiatan berdasarkan rentang 30 hari dan bulan (Januari hingga Desember). Gradasi warna biru digunakan untuk menunjukkan intensitas partisipasi, di mana warna lebih gelap mengindikasikan jumlah kegiatan yang lebih tinggi. Pada baris “Jumlah Partisipasi” di bagian bawah terlihat variasi total partisipasi dari angka 2 hingga 24, yang menandakan bahwa terdapat hari dan bulan tertentu yang menjadi puncak aktivitas mahasiswa. Visualisasi ini sangat berguna untuk mengidentifikasi periode sibuk dan tenang dalam

penyelenggaraan kegiatan, yang dapat dijadikan dasar perencanaan kalender akademik dan organisasi.

6. Top 10 Organisasi dengan Volume Kegiatan tertinggi (Bar Chart - Kanan Bawah)

Grafik batang horizontal di bagian kanan bawah menunjukkan organisasi-organisasi yang paling aktif menyelenggarakan kegiatan. BEM FT tampil sebagai organisasi dengan volume kegiatan tertinggi, mencapai sekitar 6 kegiatan dalam periode pengamatan. Posisi berikutnya ditempati oleh beberapa organisasi lain seperti ROIS FT, HIMATIKA, dan HIMATEK, yang rata-rata menyelenggarakan 3 hingga 5 kegiatan. Terdapat satu organisasi yang menonjol dengan batang berwarna hitam, kemungkinan HIMATIKA TEKKIM, yang memiliki karakteristik visual berbeda untuk penekanan. Informasi ini membantu mengidentifikasi organisasi yang paling produktif dan dapat menjadi basis evaluasi maupun apresiasi kinerja organisasi kemahasiswaan.

- Dashboard 3: Financial Analysis



1. Realisasi Trend by Month (Area Chart - Atas)

Grafik area di bagian atas menunjukkan tren realisasi dana dari Desember 2022 hingga Maret 2025. Warna area biru memperlihatkan fluktuasi nilai realisasi dari bulan ke bulan, dengan puncak tertinggi mendekati 40 juta rupiah. Pada salah satu titik, tooltip menampilkan informasi bahwa pada bulan November 2023 total realisasi dana mencapai sekitar Rp 8,19 miliar, menunjukkan adanya lonjakan yang signifikan.

Pola grafik memperlihatkan adanya periode puncak dan lembah, yang mengindikasikan ketidakkonsistenan penggunaan dana antar bulan. Visualisasi ini sangat bermanfaat untuk memantau ritme penyerapan anggaran sepanjang waktu serta mengidentifikasi periode dengan pengeluaran tinggi maupun rendah, sehingga dapat menjadi dasar pengambilan keputusan terkait perencanaan anggaran dan evaluasi pelaksanaan kegiatan.

2. Realisasi by Organization (Donut Chart - Kanan Atas)

Diagram donut pada bagian kanan atas menggambarkan proporsi realisasi dana berdasarkan organisasi. Tampak dua segmen utama mendominasi grafik dengan perbandingan 77% dan 23%, masing-masing direpresentasikan dengan warna biru dan oranye. Ketimpangan ini menunjukkan bahwa terdapat satu organisasi atau kelompok organisasi yang menyerap proporsi anggaran jauh lebih besar dibanding kelompok lainnya. Informasi ini menekankan pentingnya evaluasi efektivitas penggunaan dana antar organisasi, serta memberikan gambaran awal mengenai distribusi anggaran yang tidak merata.

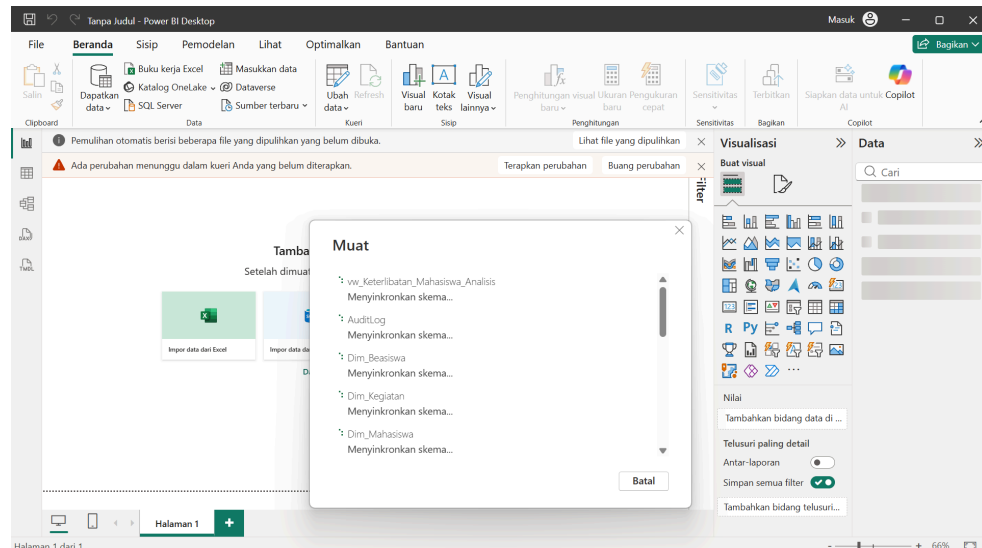
3. Realisasi Waterfall (Waterfall Chart - Kiri Bawah)

Grafik waterfall pada bagian kiri bawah menggambarkan kontribusi realisasi dana dari masing-masing organisasi. Batang berwarna abu-abu terang menandakan kontribusi positif dari tiap organisasi terhadap total realisasi, sementara batang berwarna biru gelap pada akhir grafik menunjukkan total keseluruhan mendekati Rp 100 juta. Sumbu horizontal menampilkan daftar organisasi seperti Serikat Sekerma, HMTIF FMIPA, HIMATIKA, dan lainnya. Visualisasi ini memudahkan identifikasi organisasi mana yang memberikan kontribusi realisasi terbesar dan terkecil, sehingga dapat menjadi dasar bagi evaluasi efektivitas serta prioritas pendanaan kegiatan.

4. Realisasi Projection (Line Chart - Kanan Bawah)

Bagian kanan bawah memuat grafik garis yang menampilkan proyeksi realisasi dana per tanggal. Garis hijau menunjukkan pola fluktuatif penggunaan dana dari Januari hingga Desember dengan rentang nilai antara 0 hingga 3 juta rupiah per hari. Garis tren putus-putus abu-abu mengindikasikan kecenderungan umum dari aliran dana, dengan puncak tertinggi terlihat pada bulan Desember yang hampir mencapai 3,5 juta rupiah. Informasi ini sangat berguna dalam perencanaan anggaran ke depan, terutama untuk mengantisipasi kebutuhan dana pada periode-periode tertentu yang menunjukkan kecenderungan peningkatan. Visualisasi ini juga membantu mengidentifikasi adanya pola musiman dalam penggunaan dana kegiatan.

- Connect Power BI to SQL Server



Pada tahap ini menunjukkan bahwa tidak berhasil connect, sehingga kami hanya menggunakan data excel untuk membuat dashboardnya dan menggunakan Tableau

Step 3: Security Implementation

Implementasi keamanan mencakup pembatasan hak akses (prinsip Least Privilege), penyembunyian data sensitif (DDM), dan pelacakan aktivitas (Audit Trail).

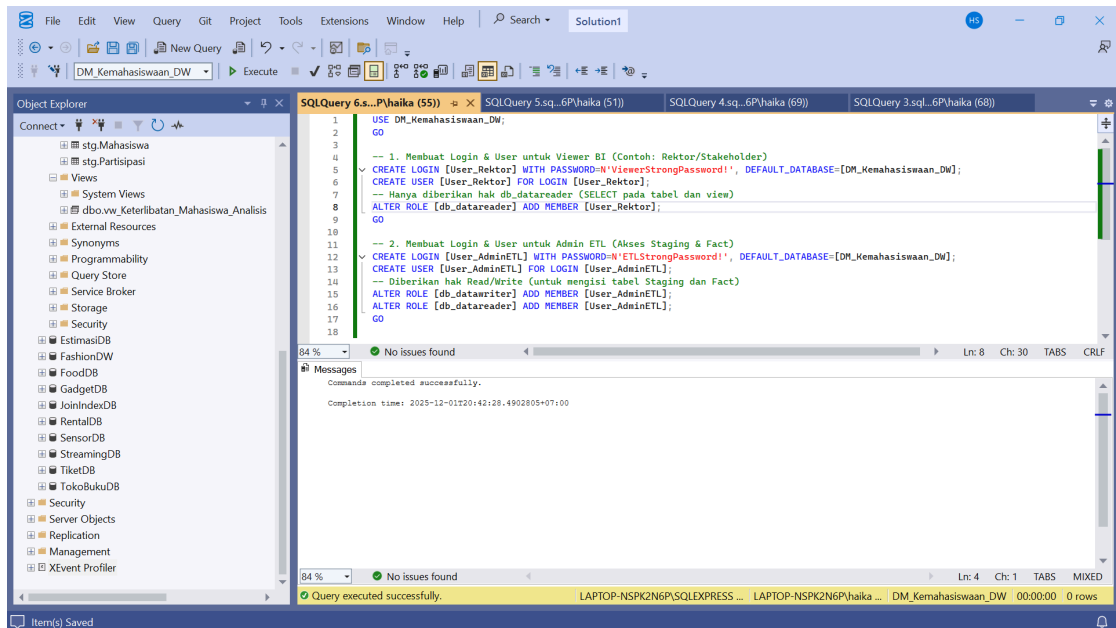
1. Create Users and Assign Roles

Kami membuat akun khusus untuk Viewer (akses baca saja) dan Admin ETL (akses baca dan tulis).

```
USE DM_Kemahasiswaan_DW;
GO
```

```
-- 1. Membuat Login & User untuk Viewer BI (Contoh: Rektor/Stakeholder)
CREATE LOGIN [User_Rektor] WITH PASSWORD=N'ViewerStrongPassword!',
DEFAULT_DATABASE=[DM_Kemahasiswaan_DW];
CREATE USER [User_Rektor] FOR LOGIN [User_Rektor];
-- Hanya diberikan hak db_datareader (SELECT pada tabel dan view)
ALTER ROLE [db_datareader] ADD MEMBER [User_Rektor];
GO
```

```
-- 2. Membuat Login & User untuk Admin ETL (Akses Staging & Fact)
CREATE LOGIN [User_AdminETL] WITH
PASSWORD=N'ETLStrongPassword!',
DEFAULT_DATABASE=[DM_Kemahasiswaan_DW];
CREATE USER [User_AdminETL] FOR LOGIN [User_AdminETL];
-- Diberikan hak Read/Write (untuk mengisi tabel Staging dan Fact)
ALTER ROLE [db_datawriter] ADD MEMBER [User_AdminETL];
ALTER ROLE [db_datareader] ADD MEMBER [User_AdminETL];
GO
```



Bagian ini menjelaskan penerapan keamanan pada Data Mart dengan fokus pada pembatasan akses pengguna dan pengaturan role. Prinsip yang digunakan adalah Least Privilege, yaitu setiap pengguna hanya diberikan hak yang benar-benar diperlukan sesuai peran mereka. Pertama, dibuat akun khusus untuk pemangku kepentingan, misalnya Rektor, menggunakan login User_Rektor. Akun ini hanya ditambahkan ke role db_datareader, sehingga hanya memiliki kemampuan untuk membaca tabel dan view tanpa bisa melakukan perubahan data. Langkah berikutnya adalah membuat akun untuk Admin ETL melalui login User_AdminETL. Akun ini diberi hak baca dan tulis dengan menambahkan user ke role db_datareader dan db_datawriter, sehingga dapat melakukan insert ke tabel staging dan fact saat proses ETL berlangsung. Pendekatan ini memastikan bahwa pengguna yang melakukan monitoring hanya dapat melihat data, sedangkan proses ETL tetap bisa berjalan dengan aman tanpa memberikan akses berlebih. Dengan konfigurasi ini, keamanan database meningkat, data sensitif terlindungi, dan akses dapat dilacak dengan lebih terstruktur.

2. Implement Data Masking

DDM diterapkan pada kolom identitas sensitif di Dim_Mahasiswa untuk menyamarkan data dari User_Rektor dan pengguna umum lainnya.

```

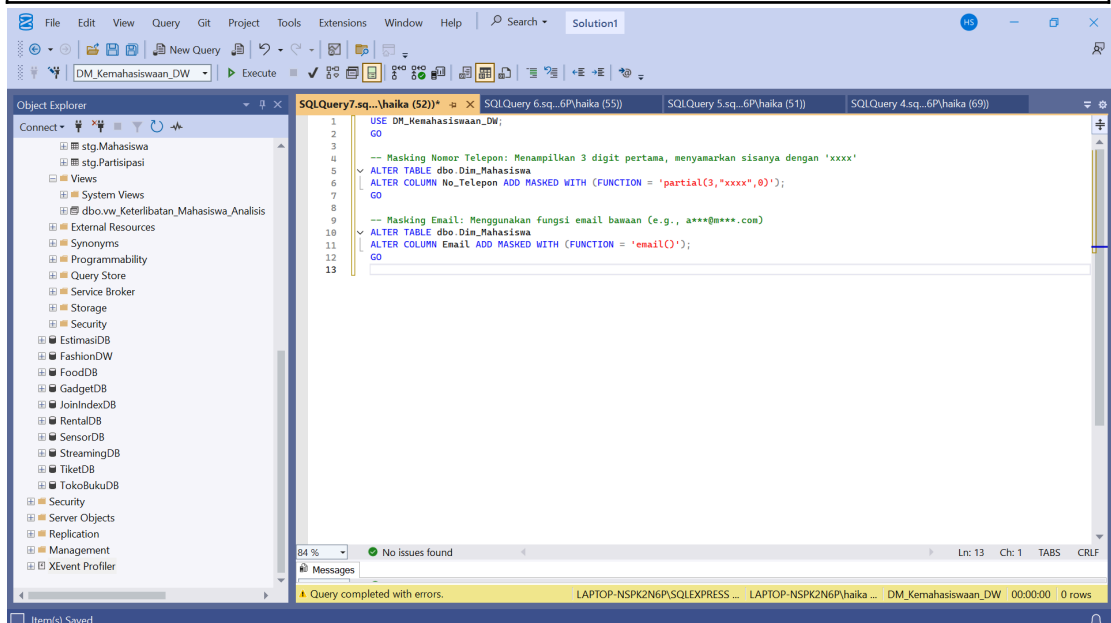
USE DM_Kemahasiswaan_DW;
GO

-- Masking Nomor Telepon: Menampilkan 3 digit pertama, menyamarkan sisanya
-- dengan 'xxxx'
ALTER TABLE dbo.Dim_Mahasiswa
ALTER COLUMN No_Telepon ADD MASKED WITH (FUNCTION =
'partial(3,"xxxx",0)');
GO

```



```
-- Masking Email: Menggunakan fungsi email bawaan (e.g., a***@m***.com)
ALTER TABLE dbo.Dim_Mahasiswa
ALTER COLUMN Email ADD MASKED WITH (FUNCTION = 'email()');
GO
```



Bagian ini menjelaskan implementasi Dynamic Data Masking (DDM) sebagai salah satu fitur keamanan untuk menyamarkan informasi sensitif pada tabel dimensi mahasiswa. Tujuan utamanya adalah melindungi privasi data seperti nomor telepon dan email, khususnya dari pengguna yang hanya memiliki hak akses baca, tetapi tidak memiliki kebutuhan untuk melihat data secara detail. Pada kolom No_Telepon, diterapkan fungsi masking `partial(3,"xxxx",0)` yang menampilkan tiga digit pertama secara jelas, sementara sisa digit diganti dengan teks "xxxx". Hal ini memungkinkan identifikasi dasar jika diperlukan, tanpa mengungkapkan informasi lengkap. Pada kolom Email, digunakan fungsi masking `email()` yang secara otomatis menghasilkan format email tersamarkan, misalnya `a***@m***.com`, sehingga struktur tetap terjaga tetapi identitas pengguna tidak terlihat. DDM bekerja secara dinamis di level query: data asli tetap tersimpan dalam bentuk lengkap, namun yang ditampilkan kepada pengguna yang tidak berhak hanyalah data hasil masking.

3. Implement Audit Trail

Kami mengimplementasikan SQL Server Audit untuk merekam aktivitas akses ke data sensitif, seperti akses `SELECT` ke `Dim_Mahasiswa`.

```
USE [master];
GO

-- 1. Membuat Server Audit
-- CATATAN: Path di bawah harus diganti ke lokasi fisik yang aman di server
produksi
CREATE SERVER AUDIT [Audit_DW_Security]
```

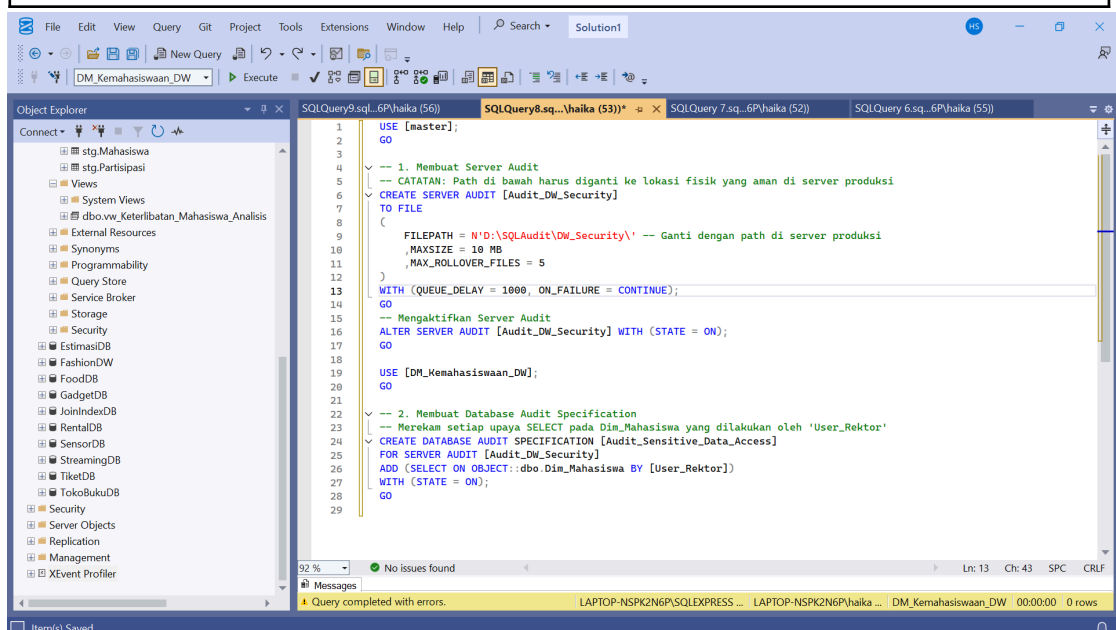
```

TO FILE
(
    FILEPATH = N'D:\SQLAudit\DW_Security\' -- Ganti dengan path di server
produksi
    ,MAXSIZE = 10 MB
    ,MAX_ROLLOVER_FILES = 5
)
WITH (QUEUE_DELAY = 1000, ON_FAILURE = CONTINUE);
GO
-- Mengaktifkan Server Audit
ALTER SERVER AUDIT [Audit_DW_Security] WITH (STATE = ON);
GO

USE [DM_Kemahasiswaan_DW];
GO

-- 2. Membuat Database Audit Specification
-- Merekam setiap upaya SELECT pada Dim_Mahasiswa yang dilakukan oleh
'User_Rektor'
CREATE DATABASE AUDIT SPECIFICATION [Audit_Sensitive_Data_Access]
FOR SERVER AUDIT [Audit_DW_Security]
ADD (SELECT ON OBJECT::dbo.Dim_Mahasiswa BY [User_Rektor])
WITH (STATE = ON);
GO

```



Bagian ini menjelaskan implementasi Audit Trail untuk memperkuat keamanan pada Data Warehouse. Mekanisme yang digunakan adalah SQL Server Audit, dimana sistem dibuat untuk merekam setiap aktivitas akses terhadap data sensitif, khususnya tabel Dim_Mahasiswa. Pertama, dibuat sebuah objek Server Audit bernama Audit_DW_Security yang menyimpan log aktivitas ke dalam file fisik di direktori khusus. Pengaturan seperti ukuran maksimum file dan jumlah file cadangan ditentukan untuk memastikan log tidak menghabiskan ruang dan tetap terkelola rapi.

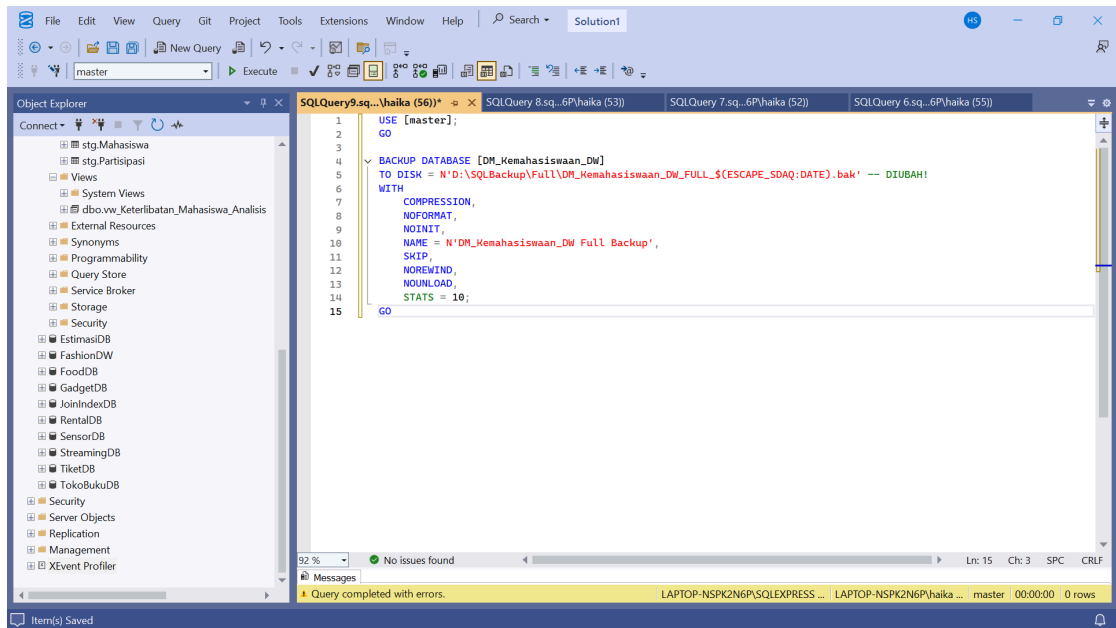
Setelah itu, audit diaktifkan sehingga server mulai mencatat peristiwa secara real time. Tahap berikutnya adalah membuat Database Audit Specification yang mendefinisikan aturan pemantauan. Pada konfigurasi ini, setiap perintah SELECT yang dilakukan oleh user tertentu, yaitu User_Rektor, pada tabel Dim_Mahasiswa akan dicatat ke dalam log audit. Dengan cara ini, institusi memiliki bukti forensik mengenai siapa yang mengakses data mahasiswa, kapan waktu akses dilakukan, dan objek apa yang diakses. Implementasi ini memastikan transparansi dan akuntabilitas, serta membantu pencegahan potensi pelanggaran privasi sesuai prinsip keamanan data dan regulasi organisasi.

Step 4: Backup and Recovery Strategy

Karena Recovery Model sudah diatur ke FULL, kami dapat menerapkan strategi backup yang mendukung Point-in-Time Recovery. Untuk T-SQL for Backup Jobs. Kami membuat job step untuk Full Backup (Mingguan) dan Log Backup (Harian/Per Jam).

1. Full Database Backup (Mingguan)

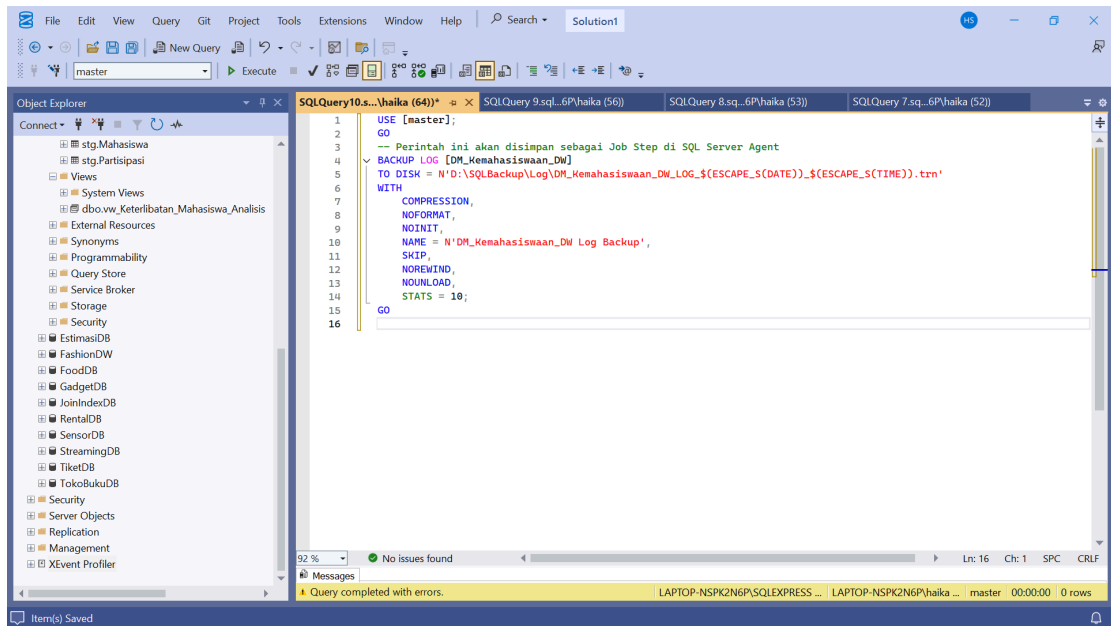
```
USE [master];
GO
-- Perintah ini akan disimpan sebagai Job Step di SQL Server Agent
BACKUP DATABASE [DM_Kemahasiswaan_DW]
TO DISK =
N'D:\SQLBackup\Full\DM_Kemahasiswaan_DW_FULL_$(ESCAPE_S(DATE)).bak'
WITH
    COMPRESSION,
    NOFORMAT,
    NOINIT,
    NAME = N'DM_Kemahasiswaan_DW Full Backup',
    SKIP,
    NOREWIND,
    NOUNLOAD,
    STATS = 10;
GO
```



Langkah ini menerapkan proses full backup mingguan untuk menjaga kelangsungan dan keamanan data pada Data Warehouse. Perintah BACKUP DATABASE menyalin seluruh isi database DM_Kemahasiswaan_DW ke media penyimpanan yang telah ditentukan, yaitu folder D:\SQLBackup\Full. Nama file backup menggunakan format dinamis dengan tanggal, sehingga setiap hasil backup disimpan sebagai file baru dan tidak menimpa cadangan sebelumnya. Opsi tambahan seperti COMPRESSION digunakan untuk mengurangi ukuran file, sedangkan parameter lain seperti NOFORMAT, NOINIT, dan SKIP memastikan media tidak diformat ulang atau diinisialisasi ulang, sehingga backup lama tetap aman tersimpan. Selain itu, pengaturan STATS = 10 berguna untuk menampilkan progress backup setiap 10% sehingga membantu proses monitoring

2. Transaction Log Backup (Harian/Per Jam)

```
USE [master];
GO
-- Perintah ini akan disimpan sebagai Job Step di SQL Server Agent
BACKUP LOG [DM_Kemahasiswaan_DW]
TO DISK =
N'D:\SQLBackup\Log\DM_Kemahasiswaan_DW_LOG_$(ESCAPE_S(DATE))_$(
ESCAPE_S(TIME)).trn'
WITH
    COMPRESSION,
    NOFORMAT,
    NOINIT,
    NAME = N'DM_Kemahasiswaan_DW Log Backup',
    SKIP,
    NOREWIND,
    NOUNLOAD,
    STATS = 10;
GO
```



Kode ini menerapkan backup Transaction Log secara berkala untuk mendukung Point-in-Time Recovery. Perintah BACKUP LOG digunakan untuk mencadangkan hanya perubahan data yang terjadi sejak backup terakhir, sehingga ukuran file backup lebih kecil dan proses lebih cepat dibanding full backup. Hasil backup disimpan dalam folder D:\SQLBackup\Log dengan format nama file yang mengandung tanggal dan waktu ($(\text{ESCAPE_S}(\text{DATE}))_(\text{ESCAPE_S}(\text{TIME}))$). Hal ini memastikan setiap file log tersimpan unik tanpa saling menimpa, sehingga administrator dapat memilih titik pemulihan yang tepat jika terjadi kegagalan. Penggunaan opsi COMPRESSION membantu mengurangi ukuran file, sedangkan parameter seperti NOFORMAT, NOINIT, dan SKIP memastikan media tidak di-reset dan tetap aman menyimpan backup sebelumnya. Informasi kemajuan proses ditampilkan melalui STATS = 10, yang menunjukkan progres setiap 10%.

Step 5: User Acceptance Testing (UAT)

Sub-Step	Tujuan Utama	Jenis Aktivitas	Kebutuhan Kode
1. Create Test Cases	Mendefinisikan <i>skenario validasi</i> .	Dokumentasi dan perumusan kueri (T-SQL SELECT) untuk validasi data.	Skrip SQL SELECT untuk validasi data (DQ Checks).
2. Conduct UAT Sessions	Memvalidasi fungsionalitas dan tampilan <i>dashboard</i> oleh <i>end-users</i> (Stakeholders).	Pertemuan, <i>demonstrasi live</i> , dan pencatatan <i>feedback</i> .	Tidak ada kode baru.
3. Performance Testing	Mengukur <i>Query Response Time</i> dan <i>ETL Execution Time</i> .	Menjalankan kueri kompleks sambil mengaktifkan SET	Skrip SQL SELECT untuk <i>benchmarking</i> .

		STATISTICS TIME ON dan SET STATISTICS IO ON.	
4. Refinement	Memperbaiki dan mengoptimalkan.	Implementasi perubahan <i>indexing</i> , <i>stored procedure</i> , atau <i>bug fix</i> .	Kode DDL/DML Modifikasi (Contoh: CREATE INDEX, ALTER PROCEDURE).

UAT adalah fase kritis untuk memastikan Data Warehouse (DW) dan Data Mart baru tidak hanya berfungsi secara teknis tetapi juga akurat, aman, dan memenuhi kebutuhan fungsional pengguna akhir (misalnya, Rektorat, Kepala Program Studi, dan Tim Keuangan).

1. Create Test Cases

Kami mengembangkan test case yang berfokus pada tiga area utama: Integritas Data, Fungsionalitas ETL, dan Keamanan Data.

Tujuan: Memastikan data yang baru dimuat ke Fact_Capaian_Prestasi dan Fact_Penerima_Beasiswa akurat, tidak duplikat, dan terhubung dengan benar ke Dimensi yang Aktif (IsCurrent = 1).

1. Data Quality Checks (DQ_001, DQ_002, DQ_004)

```
-- File: 08_Data_Quality_Checks_M3.sql
USE DM_Kemahasiswaan_DW;
GO

-----
-- DQ_001: Completeness - Cek NULL Foreign Keys di Fact_Partisipasi_Kegiatan
-- (Fact yang kolom FK-nya didefinisikan sebagai NOT NULL, kecuali
Organisasi_SK)
-----
PRINT '--- Checking DQ_001: NULL Foreign Keys in Fact_Partisipasi_Kegiatan
---';
-- Kolom SK yang NOT NULL di Fact_Partisipasi_Kegiatan: Tanggal_SK,
Mahasiswa_SK, Kegiatan_SK.
-- Organisasi_SK diizinkan NULL, tetapi tetap dicek jika ada kebutuhan bisnis
SELECT
    'Fact_Partisipasi_Kegiatan' AS TableName,
    COUNT(*) AS NullKeyCount,
    SUM(CASE WHEN Mahasiswa_SK IS NULL THEN 1 ELSE 0 END) AS
NullMahasiswa_SK, -- Mahasiswa_SK (NOT NULL)
    SUM(CASE WHEN Kegiatan_SK IS NULL THEN 1 ELSE 0 END) AS
NullKegiatan_SK, -- Kegiatan_SK (NOT NULL)
    SUM(CASE WHEN Tanggal_SK IS NULL THEN 1 ELSE 0 END) AS
```

```

NullTanggal_SK, -- Tanggal_SK (NOT NULL)
SUM(CASE WHEN Organisasi_SK IS NULL THEN 1 ELSE 0 END) AS
NullOrganisasi_SK -- Organisasi_SK (NULLABLE)
FROM dbo.Fact_Partisipasi_Kegiatan
HAVING
SUM(CASE WHEN Mahasiswa_SK IS NULL THEN 1 ELSE 0 END) > 0 OR
SUM(CASE WHEN Kegiatan_SK IS NULL THEN 1 ELSE 0 END) > 0 OR
SUM(CASE WHEN Tanggal_SK IS NULL THEN 1 ELSE 0 END) > 0 OR
SUM(CASE WHEN Organisasi_SK IS NULL THEN 1 ELSE 0 END) > 0;
GO

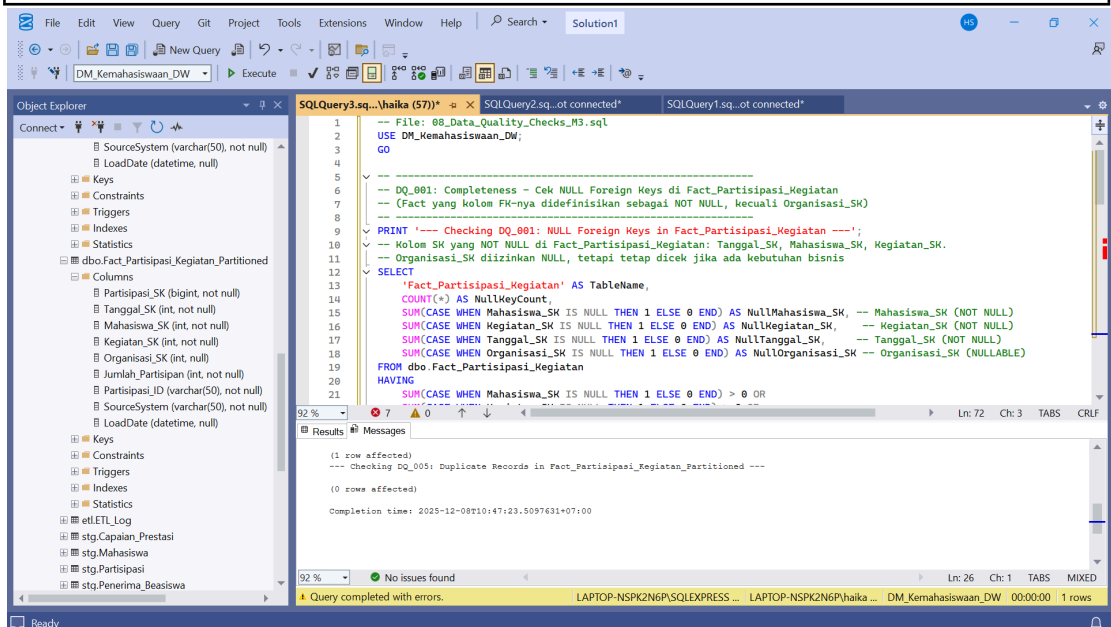
-----
-- DQ_002: Consistency - Cek Orphan Records di stg.Capaian_Prestasi
-- (Mengecek record staging yang menunjuk ke dimensi yang tidak ada)
-----
PRINT '--- Checking DQ_002: Orphan Records in stg.Capaian_Prestasi ---';
-- ASUMSI: Dim_Student dan Dim_Achievement/Dim_Prestasi ada, tetapi tidak
ditampilkan kolomnya.
-- ASUMSI: Staging menggunakan Natural Key (misalnya Mahasiswa_ID,
Prestasi_ID)
SELECT
'stg.Capaian_Prestasi' AS TableName,
COUNT(s.SourceID) AS OrphanRecordsCount -- Asumsi kolom ID sumber di
Staging
FROM stg.Capaian_Prestasi s
-- ASUMSI: Ada Dim_Mahasiswa yang berisi Mahasiswa_SK
LEFT JOIN dbo.Dim_Mahasiswa dm ON s.Mahasiswa_ID = dm.Mahasiswa_ID --
Pengecekan Natural Key Mahasiswa
-- ASUMSI: Dim_Prestasi ada
LEFT JOIN dbo.Dim_Prestasi dp ON s.Prestasi_ID = dp.Prestasi_ID -- Pengecekan
Natural Key Prestasi
WHERE dm.Mahasiswa_ID IS NULL OR dp.Prestasi_ID IS NULL;
GO

-----
-- DQ_004: Consistency - Cek Fact yang Terhubung ke Record SCD Type 2 Lama
-- (Mengecek apakah ada data partisipasi yang masuk ke record Dim_Mahasiswa
lama)
-----
PRINT '--- Checking DQ_004: Fact Linked to Inactive SCD Records ---';
-- ASUMSI: Dim_Mahasiswa adalah Dimensi SCD Type 2 dengan kolom IsCurrent
SELECT
'Fact_Partisipasi_Kegiatan' AS TableName,
COUNT(f.Partisipasi_SK) AS InactiveSCDLinkCount
FROM dbo.Fact_Partisipasi_Kegiatan f
INNER JOIN dbo.Dim_Mahasiswa s ON f.Mahasiswa_SK = s.Mahasiswa_SK
WHERE s.IsCurrent = 0;
GO

```

```
-- DQ_005: Duplikasi - Cek Duplikasi di Fact_Partisipasi_Kegiatan_Partitioned
-- (Fact unik berdasarkan Partisipasi_ID + SourceSystem)
```

```
PRINT '--- Checking DQ_005: Duplicate Records in
Fact_Partisipasi_Kegiatan_Partitioned ---';
-- Kolom Natural Key di Fact_Partisipasi_Kegiatan_Partitioned: Partisipasi_ID dan
SourceSystem
SELECT
    Partisipasi_ID,
    SourceSystem,
    COUNT(*) AS DuplicateCount
FROM dbo.Fact_Partisipasi_Kegiatan_Partitioned
GROUP BY Partisipasi_ID, SourceSystem
HAVING COUNT(*) > 1;
GO
```



Bagian ini bertujuan untuk menguji kualitas data yang telah dimuat ke dalam Data Mart. Serangkaian test case dibuat untuk memastikan bahwa data memenuhi tiga prinsip utama, yaitu: kelengkapan (completeness), konsistensi (consistency), dan tidak terjadi duplikasi (uniqueness). Seluruh kode dijalankan pada database DM_Kemahasiswaan_DW, sehingga seluruh pengecekan dilakukan terhadap tabel fact dan dimensi aktif yang menjadi bagian dari sistem data warehouse.

1. DQ_001 – Completeness: Pengecekan NULL Foreign Key

Query pertama berfungsi untuk mendeteksi apakah terdapat nilai NULL pada foreign key kritis di tabel Fact_Partisipasi_Kegiatan. Kolom seperti Mahasiswa_SK, Kegiatan_SK, dan Tanggal_SK didefinisikan sebagai NOT NULL, sehingga keberadaan nilai kosong akan menyebabkan integritas hubungan antar tabel gagal. Query menghitung total baris serta jumlah NULL pada masing-masing kolom. Bagian HAVING digunakan untuk hanya menampilkan hasil jika terdapat minimal satu nilai NULL. Hal ini membantu

tim memastikan bahwa setiap baris fact benar-benar memiliki referensi ke dimensi terkait, sehingga laporan analitik tidak mengandung data yang hilang atau rusak.

2. DQ_002 – Consistency: Pengecekan Orphan Records di Staging

Pengecekan kedua memvalidasi konsistensi data antara staging (stg.Capaian_Prestasi) dan tabel dimensi. Query melakukan LEFT JOIN antara staging dan Dim_Mahasiswa serta Dim_Prestasi berdasarkan natural key. Baris yang gagal mendapatkan pasangan akan teridentifikasi sebagai orphan records, yaitu data staging yang menunjuk ke entitas yang tidak ada di dimensi. Kondisi ini berbahaya karena data tersebut akan gagal dimuat ke fact table dan dapat menyebabkan hilangnya histori capaian prestasi mahasiswa. Output berupa jumlah orphan menjadi indikator apakah proses ETL sebelumnya perlu diperbaiki.

3. DQ_004 – Consistency: Deteksi Link ke Record SCD Lama

Pengecekan ketiga berfokus pada dimensi bertipe SCD Type 2, di mana perubahan informasi mahasiswa disimpan dalam versi historis. Query ini mengecek apakah fact table masih mengarah ke record dimensi yang sudah tidak aktif (IsCurrent = 0). Jika ditemukan, hal ini mengindikasikan bahwa proses ETL gagal memperbarui foreign key saat menjalankan dimensi terbaru. Kondisi ini dapat menyebabkan analisis berbasis status terkini menjadi tidak akurat. Oleh karena itu, hasil dari query ini digunakan untuk memverifikasi bahwa hubungan fact selalu menunjuk ke versi yang aktif.

4. DQ_005 – Uniqueness: Deteksi Duplikasi Data

Pengecekan terakhir bertujuan untuk mendeteksi duplikasi pada tabel Fact_Partisipasi_Kegiatan_Partitioned. Natural key dari fact ini adalah kombinasi Partisipasi_ID dan SourceSystem, sehingga kombinasi keduanya harus unik. Query melakukan GROUP BY berdasarkan kedua kolom tersebut dan menampilkan baris yang memiliki jumlah lebih dari satu (HAVING COUNT(*) > 1). Temuan duplikasi sangat penting karena dapat menyebabkan agregasi yang salah pada laporan dashboard, misalnya perhitungan jumlah partisipasi kegiatan menjadi berlipat. Jika ditemukan duplikat, tim dapat melakukan pembersihan data pada level staging atau memperbaiki logika ETL.

2. ETL Fungsionalitas (Simulasi ETL_002 - SCD Type 2)

```
-- File: 07_ETL_Simulasi_SCD_M3.sql
USE DM_Kemahasiswaan_DW;
GO
```

```

-- Asumsi: Dim_Program sudah terisi, Dim_Mahasiswa awal kosong.

-- Step 1: Insert data Mahasiswa Awal di Staging
-- Ditambahkan kolom Fakultas dan LoadDate (asumsi LoadDate NOT NULL)
INSERT INTO stg.Mahasiswa (NIM, Nama_Mahasiswa, Fakultas, Program_Studi,
Tahun_Masuk, Status_Mahasiswa, LoadDate)
VALUES ('123450001', 'Budi Santoso', 'Teknik', 'IF', 2022, 'Aktif', GETDATE());
GO

-- Step 2: Jalankan Load Dimensi (Initial Load)
-- Asumsi usp_Load_Dim_Mahasiswa sudah ada dan berfungsi.
EXEC dbo.usp_Load_Dim_Mahasiswa;
GO

-- Cek Hasil Awal (Mahasiswa A harus IsCurrent = 1)
SELECT Mahasiswa_SK, NIM, Nama_Mahasiswa, Status_Mahasiswa, IsCurrent,
EffectiveDate, ExpiryDate
FROM dbo.Dim_Mahasiswa WHERE NIM = '123450001';
GO

-----

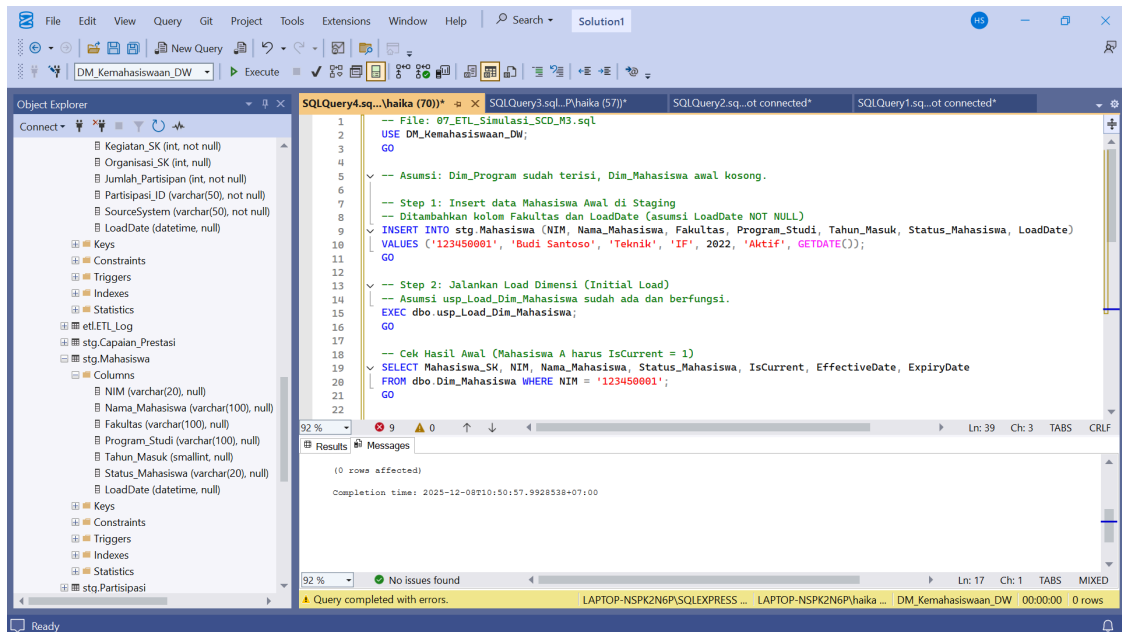
-- ETL_002: Skenario Perubahan (SCD Type 2 Trigger)
-- Perubahan Status: 'Aktif' -> 'Lulus'
-----

-- Step 3: Update data Mahasiswa A di Staging (perubahan status)
UPDATE stg.Mahasiswa SET Status_Mahasiswa = 'Lulus', LoadDate =
GETDATE() WHERE NIM = '123450001';
GO

-- Step 4: Jalankan Load Dimensi (Perubahan)
EXEC dbo.usp_Load_Dim_Mahasiswa;
GO

-- Cek Hasil Akhir (Verifikasi ETL_002)
SELECT Mahasiswa_SK, NIM, Nama_Mahasiswa, Status_Mahasiswa, IsCurrent,
EffectiveDate, ExpiryDate
FROM dbo.Dim_Mahasiswa WHERE NIM = '123450001' ORDER BY
Mahasiswa_SK DESC;
GO

```

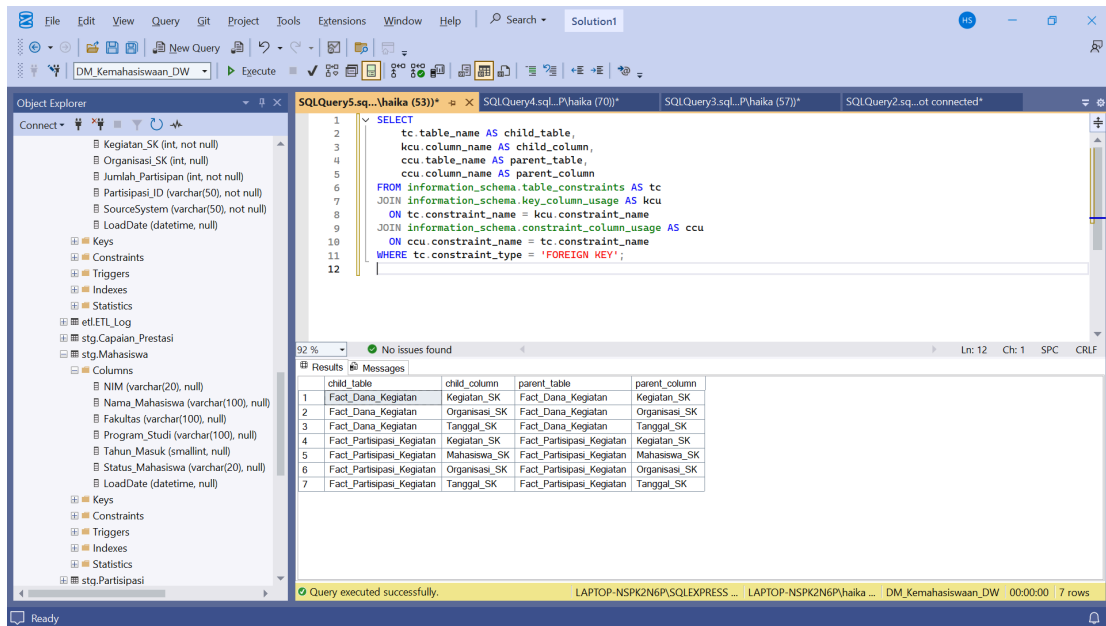


Kode ini mensimulasikan proses ETL yang memicu mekanisme Slowly Changing Dimension Type 2 (SCD Type 2) pada Dim_Mahasiswa. Awalnya, data mahasiswa baru dimasukkan ke staging lengkap dengan atribut utama dan timestamp LoadDate. Prosedur usp_Load_Dim_Mahasiswa kemudian dijalankan sebagai initial load sehingga satu record mahasiswa tercatat di dimensi dengan status “Aktif” dan ditandai sebagai baris aktif (IsCurrent = 1). Setelah itu, dilakukan perubahan status di staging dari “Aktif” menjadi “Lulus”, yang bertujuan memicu logika historisasi. Ketika load dimensi dijalankan kembali, proses SCD Type 2 akan membuat baris baru untuk mahasiswa yang lulus, sementara record lama di-set menjadi tidak aktif (IsCurrent = 0) dengan update pada kolom EffectiveDate dan ExpiryDate. Query akhir yang menampilkan hasil berdasarkan Mahasiswa_SK secara descending memastikan bahwa versi terbaru muncul paling atas sebagai status aktif, sedangkan versi sebelumnya tetap tersimpan sebagai riwayat.

Step 6: Documentation

1. System Architecture Document

```
SELECT
    tc.table_name AS child_table,
    kcu.column_name AS child_column,
    ccu.table_name AS parent_table,
    ccu.column_name AS parent_column
FROM information_schema.table_constraints AS tc
JOIN information_schema.key_column_usage AS kcu
    ON tc.constraint_name = kcu.constraint_name
JOIN information_schema.constraint_column_usage AS ccu
    ON ccu.constraint_name = tc.constraint_name
WHERE tc.constraint_type = 'FOREIGN KEY';
```



Kode pada bagian dokumentasi ini digunakan untuk menghasilkan daftar relasi antar tabel berdasarkan Foreign Key yang ada dalam database. Query memanfaatkan metadata dari information_schema, yaitu kumpulan tabel sistem yang menyimpan definisi struktur database. Melalui join pada table_constraints, key_column_usage, dan constraint_column_usage, query dapat mengekstraksi informasi lengkap mengenai tabel anak, kolom yang menjadi Foreign Key, serta tabel dan kolom induk yang dirujuk. Hasil output berupa daftar pasangan child-parent yang memperlihatkan bagaimana fact dan dimension saling terhubung melalui kunci asing.

2. Data Dictionary (Update dari Misi 1)

A. Fact Tables (Tabel Fakta)

Fact_Partispasi_Kegiatan

Kolom	Tipe Data	PK/FK	Deskripsi	Busines Rule
Partispasi_SK	INT	PK	Surrogate Key unik	Auto-increment, NOT NULL
Mahasiswa_SK	INT	FK	Key ke Dim_Mahasiswa	NOT NULL
Kegiatan_SK	INT	FK	Key ke Dim_Kegiatan	NOT NULL
Organisasi_SK	INT	FK	Key ke Dim_Organisasi	Dapat NULL
Tanggal_SK	INT	FK	Key ke Dim_Tanggal	NOT NULL
Jumlah_Partisipan	INT	Measure	Metrik partisipasi	Nilai selalu 1

				(Additif)
--	--	--	--	-----------

Tabel Fact_Partisipasi_Kegiatan merupakan tabel fakta yang mencatat partisipasi mahasiswa dalam kegiatan dengan enam kolom utama. Partisipasi_SK sebagai Primary Key (INT) yang bersifat auto-increment dan NOT NULL untuk identifikasi unik setiap record. Tabel Fact_Partisipasi_Kegiatan memiliki empat Foreign Key yang menghubungkan ke dimensi terkait: Mahasiswa_SK ke Dim_Mahasiswa (NOT NULL), Kegiatan_SK ke Dim_Kegiatan (NOT NULL), Organisasi_SK ke Dim_Organisasi (dapat NULL karena tidak semua kegiatan diselenggarakan organisasi), dan Tanggal_SK ke Dim_Tanggal (NOT NULL). Kolom Jumlah_Partisipan (INT) merupakan measure yang selalu bernilai 1 dan bersifat aditif untuk menghitung total partisipasi. Struktur ini memungkinkan analisis partisipasi mahasiswa berdasarkan dimensi mahasiswa, kegiatan, organisasi, dan waktu secara multidimensional.

B. Fact_Capaian_Prestasi

Kolom	Tipe Data	PK/FK	Deskripsi	Business Rule
Prestasi_Fact_SK	INT	PK	Surrogate Key unik untuk setiap capaian	Auto-increment, NOT NULL
Mahasiswa_SK	INT	FK	Key ke Dim_Mahasiswa	NOT NULL
Prestasi_SK	INT	FK	Key ke Dim_Prestasi	NOT NULL
Tanggal_SK	INT	FK	Key ke Dim_Tanggal (Tanggal capaian)	NOT NULL
Jumlah_Penghargaan	INT	Measure	Total penghargaan yang dicapai	Nilai selalu 1 (Additif)
Poin_Prestasi	DECIMAL (5,2)	Measure	Nilai bobot untuk Prestasi	Semi-Additif

Tabel Fact_Capaian_Prestasi merupakan tabel fakta yang mencatat prestasi mahasiswa dengan enam kolom. Prestasi_Fact_SK sebagai Primary Key (INT) yang auto-increment dan NOT NULL untuk identifikasi unik setiap capaian prestasi. Tabel ini memiliki tiga Foreign Key: Mahasiswa_SK ke Dim_Mahasiswa (NOT NULL), Prestasi_SK ke Dim_Prestasi (NOT NULL), dan Tanggal_SK ke Dim_Tanggal yang mencatat tanggal capaian prestasi (NOT NULL). Terdapat dua kolom measure: Jumlah_Penghargaan (INT) yang selalu bernilai 1 dan bersifat aditif untuk menghitung total penghargaan, serta Poin_Prestasi (DECIMAL 5,2) yang merekam nilai bobot prestasi dan bersifat semi-aditif karena poin tidak selalu dapat dijumlahkan

langsung dalam semua konteks analisis. Struktur ini memungkinkan analisis prestasi mahasiswa berdasarkan dimensi mahasiswa, jenis prestasi, dan waktu pencapaian.

C. Fact_Dana_Kegiatan

Kolom	Tipe Data	PK/FK	Deskripsi	Business Rule
Dana_SK	INT	PK	Surrogate Key untuk transaksi dana	Auto-increment, NOT NULL
Kegiatan_SK	INT	FK	Key ke Dim_Kegiatan	NOT NULL
Tanggal_SK	INT	FK	Key ke Dim_Tanggal (Tanggal Realisasi)	NOT NULL
Jumlah_Pengajuan	INT	Measure	Total anggaran yang diajukan	Additif, Mata uang Rupiah
Jumlah_Realisasi	INT	Measure	Total dana yang direalisasikan	Additif, Digunakan untuk efisiensi

Tabel Fact_Dana_Kegiatan merupakan tabel fakta yang mencatat transaksi keuangan untuk setiap kegiatan dengan lima kolom. Dana_SK sebagai Primary Key (INT) yang auto-increment dan NOT NULL untuk identifikasi unik setiap transaksi dana. Tabel ini memiliki dua Foreign Key: Kegiatan_SK ke Dim_Kegiatan (NOT NULL) untuk mengidentifikasi kegiatan yang didanai, dan Tanggal_SK ke Dim_Tanggal (NOT NULL) yang mencatat tanggal realisasi dana. Terdapat dua kolom measure yang keduanya bertipe INT dan bersifat aditif: Jumlah_Pengajuan yang merekam total anggaran yang diajukan dalam mata uang Rupiah, dan Jumlah_Realisasi yang mencatat total dana yang benar-benar direalisasikan, di mana perbandingan kedua measure ini digunakan untuk mengukur tingkat efisiensi penggunaan dana kegiatan. Struktur ini memungkinkan analisis keuangan kegiatan berdasarkan dimensi kegiatan dan waktu untuk evaluasi pengelolaan anggaran.

D. Fact_Penerima_Basiswa

Kolom	Tipe Data	PK/FK	Deskripsi	Business Rule
Beasiswa_Fact_SK	INT	PK	Surrogate Key untuk penerima beasiswa	Auto-increment, NOT NULL
Mahasiswa_SK	INT	FK	Key ke Dim_Mahasiswa	NOT NULL
Beasiswa_SK	INT	FK	Key ke Dim_Beasiswa	NOT NULL
Tanggal_SK	INT	FK	Key ke Dim_Tanggal (Tanggal Mulai/Penerimaan)	NOT NULL
Jumlah_Penerima	INT	Measure	Indikator jumlah penerima beasiswa	Nilai selalu 1 (Additif)

Tabel Fact_Penerima_Beasiswa merupakan tabel fakta yang mencatat data penerima beasiswa dengan lima kolom. Beasiswa_Fact_SK sebagai Primary Key (INT) yang auto-increment dan NOT NULL untuk identifikasi unik setiap record penerima beasiswa. Tabel ini memiliki tiga Foreign Key yang semuanya NOT NULL: Mahasiswa_SK ke Dim_Mahasiswa untuk mengidentifikasi mahasiswa penerima, Beasiswa_SK ke Dim_Beasiswa untuk menentukan jenis beasiswa yang diterima, dan Tanggal_SK ke Dim_Tanggal yang mencatat tanggal mulai/penerimaan beasiswa. Kolom Jumlah_Penerima (INT) merupakan measure yang bersifat aditif dengan nilai selalu 1, berfungsi sebagai indikator untuk menghitung total jumlah penerima beasiswa dalam berbagai analisis dimensional. Struktur ini memungkinkan analisis distribusi beasiswa berdasarkan dimensi mahasiswa, jenis beasiswa, dan periode waktu untuk evaluasi program bantuan mahasiswa.

3. ETL Documentation

- ETL Package

```
USE DM_Kemahasiswaan_DW;
GO

/* 0. CREATE SCHEMA ETL (PACKAGE ROOT) */
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = 'etl')
    EXEC('CREATE SCHEMA etl');
GO
```

```

/* 1. CREATE ETL LOG TABLE */
IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = 'ETL_Log')
BEGIN
    CREATE TABLE etl.ETL_Log (
        LogID INT IDENTITY(1,1) PRIMARY KEY,
        ETL_Procedure VARCHAR(200),
        Status VARCHAR(20),
        ErrorMessage VARCHAR(MAX) NULL,
        LogDate DATETIME DEFAULT GETDATE()
    );
END;
GO

/* 2. PROCEDURE – LOAD DIM PROGRAM (SCD Type 1) */

IF EXISTS (SELECT 1 FROM sys.objects WHERE name = 'Load_DimProgram')
    DROP PROCEDURE etl.Load_DimProgram;
GO

CREATE PROCEDURE etl.Load_DimProgram
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @ProcName VARCHAR(200) = 'Load_DimProgram';

    BEGIN TRY
        BEGIN TRANSACTION;

        -- Extract & Transform (Asumsi stg.Program sudah ada)
        SELECT
            ProgramCode,
            UPPER(LTRIM(RTRIM(ProgramName))) AS ProgramName,
            UPPER(LTRIM(RTRIM(Faculty))) AS Faculty
        INTO #Temp_Program
        FROM stg.Program
        WHERE ProgramCode IS NOT NULL;

        -- Load (MERGE untuk INSERT dan UPDATE - SCD Type 1)
        MERGE INTO dbo.Dim_Program AS T
        USING #Temp_Program AS S
        ON T.ProgramCode = S.ProgramCode
        WHEN MATCHED AND (
            T.ProgramName <> S.ProgramName OR T.Faculty <> S.Faculty -- Cek
            perubahan
        ) THEN
            UPDATE SET
                T.ProgramName = S.ProgramName,
                T.Faculty = S.Faculty
    
```



```

WHEN NOT MATCHED BY TARGET THEN
    INSERT (ProgramCode, ProgramName, Faculty)
    VALUES (S.ProgramCode, S.ProgramName, S.Faculty);

DROP TABLE #Temp_Program;

INSERT INTO etl.ETL_Log (ETL_Procedure, Status)
VALUES (@ProcName, 'SUCCESS');

COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;

    INSERT INTO etl.ETL_Log (ETL_Procedure, Status, ErrorMessage)
    VALUES (@ProcName, 'FAILED', ERROR_MESSAGE());
END CATCH;
END;
GO

/* 3. PROCEDURE – LOAD DIM STUDENT (SCD Type 2) */
IF EXISTS (SELECT 1 FROM sys.objects WHERE name = 'Load_DimStudent')
    DROP PROCEDURE etl.Load_DimStudent;
GO

CREATE PROCEDURE etl.Load_DimStudent
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @ProcName VARCHAR(200) = 'Load_DimStudent';

    -- Kolom yang memicu SCD Type 2: StudentName, ProgramCode, Status
    BEGIN TRY
        BEGIN TRANSACTION;

        -- 1. Expire old records (Jika ada perubahan pada kolom pemicu)
        UPDATE T
        SET
            T.ExpiryDate = GETDATE(),
            T.IsCurrent = 0
        FROM dbo.Dim_Student T
        INNER JOIN stg.Student S ON T.NIM = S.NIM
        WHERE T.IsCurrent = 1
        AND (
            T.StudentName <> S.StudentName
            OR T.ProgramCode <> S.ProgramCode
            OR T.Status <> S.Status
        );
    
```

```

-- 2. Insert new/updated records (Termasuk data baru dan data yang berubah)
INSERT INTO dbo.Dim_Student (
    NIM, StudentName, Gender, BirthDate, EnrollmentDate, ProgramCode,
    ProgramName, Faculty, EntryYear, Status, EffectiveDate, IsCurrent
)
SELECT
    S.NIM,
    UPPER(TRIM(S.StudentName)),
    S.Gender, S.BirthDate, S.EnrollmentDate,
    S.ProgramCode,
    P.ProgramName, -- Lookup dari Dim_Program
    P.Faculty,    -- Lookup dari Dim_Program
    YEAR(S.EnrollmentDate),
    S.Status,
    GETDATE(), -- EffectiveDate
    1          -- IsCurrent
FROM stg.Student S
LEFT JOIN dbo.Dim_Program P ON S.ProgramCode = P.ProgramCode
WHERE NOT EXISTS (
    SELECT 1
    FROM dbo.Dim_Student D
    WHERE D.NIM = S.NIM AND D.IsCurrent = 1
);

INSERT INTO etl.ETL_Log (ETL_Procedure, Status)
VALUES (@ProcName, 'SUCCESS');

COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;

    INSERT INTO etl.ETL_Log (ETL_Procedure, Status, ErrorMessage)
    VALUES (@ProcName, 'FAILED', ERROR_MESSAGE());
END CATCH;
END;
GO

/* 4. MASTER PROCEDURE – MENJALANKAN SEMUA */
IF EXISTS (SELECT 1 FROM sys.objects WHERE name =
'Master_ETL_Mahasiswa')
    DROP PROCEDURE etl.Master_ETL_Mahasiswa;
GO

CREATE PROCEDURE etl.Master_ETL_Mahasiswa
AS
BEGIN
    PRINT 'Memulai ETL Data Mart Kemahasiswaan...';

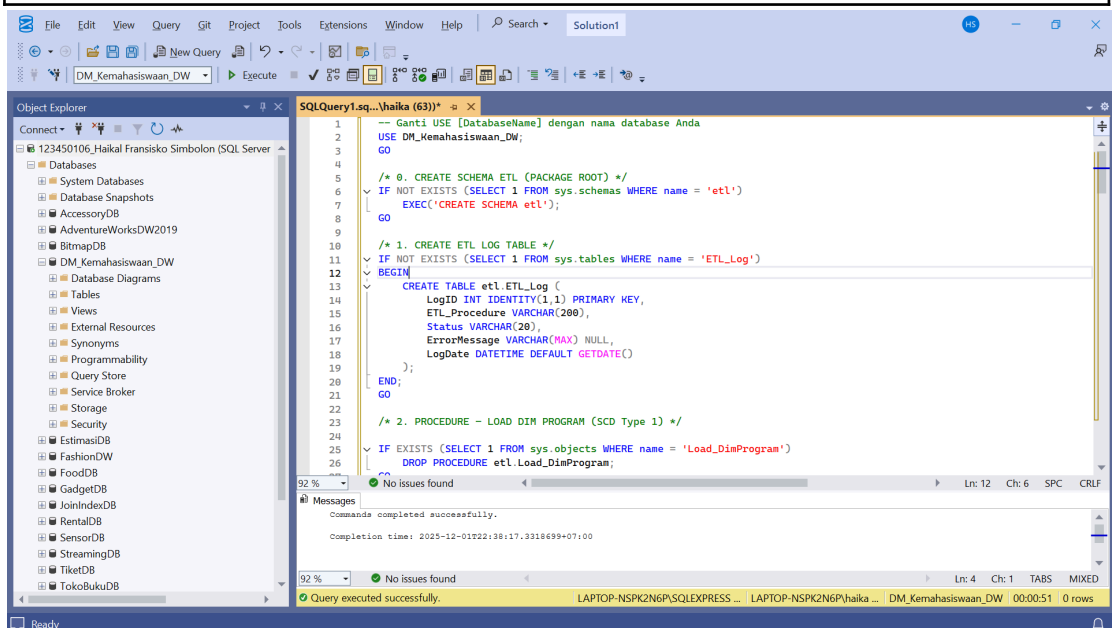
```

```
-- Step 1: Load Dimensions  
EXEC etl.Load_DimProgram;  
EXEC etl.Load_DimStudent;
```

-- TO DO: Tambahkan prosedur untuk Load Fact Tables (e.g.,
Load_FactActivityParticipation)

```
PRINT 'ETL Dimensi Kemahasiswaan selesai';  
END;  
GO
```

```
/* END OF PACKAGE */
```



Paket ETL Documentation membangun proses pemuatan data yang terstruktur dengan membuat schema etl, tabel log, serta rangkaian prosedur untuk memproses dimensi. Schema etl berfungsi sebagai wadah terpusat, sementara tabel etl.ETL_Log digunakan untuk mencatat status dan pesan kesalahan setiap eksekusi sebagai audit trail. Prosedur Load_DimProgram menerapkan SCD Type 1 menggunakan perintah MERGE, sehingga perubahan atribut seperti nama program atau fakultas langsung diperbarui tanpa menyimpan versi lama, dan seluruh proses dibungkus dalam transaksi untuk menjaga konsistensi. Sebaliknya, prosedur Load_DimStudent mengimplementasikan SCD Type 2, yaitu mendeteksi perubahan pada atribut mahasiswa, menandai record lama sebagai tidak aktif (IsCurrent = 0) dan memasukkan record baru dengan EffectiveDate sebagai riwayat. Lookup ke Dim_Program memastikan data fakultas dan program tetap sinkron. Semua keberhasilan dan kegagalan dicatat ke tabel log. Prosedur terakhir, Master_ETL_Mahasiswa, bertindak sebagai orkestrator untuk menjalankan seluruh prosedur ETL secara berurutan dan siap diperluas untuk memuat tabel fact, sehingga

keseluruhan paket ETL menjadi modular, terdokumentasi, dan siap dijalankan otomatis melalui SQL Server Agent.

- ETL Script

```
USE DM_Kemahasiswaan_DW;
GO

/* 0. CREATE TABLE LOG (JIKA BELUM ADA) */
IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name =
'ETL_Log')
BEGIN
    CREATE TABLE ETL_Log (
        LogID INT IDENTITY(1,1) PRIMARY KEY,
        ETL_Procedure VARCHAR(200),
        Status VARCHAR(20),
        ErrorMessage VARCHAR(MAX) NULL,
        LogDate DATETIME DEFAULT GETDATE()
    );
END;
GO

/* 1. ETL DIM_PROGRAM (SCD Type 1: Update jika Nama/Fakultas
berubah) */
BEGIN TRY
    PRINT 'Memulai ETL DIM_PROGRAM (SCD Type 1)...';
    BEGIN TRANSACTION;

    -- Extract
    SELECT
        ProgramCode,
        ProgramName,
        Faculty
    INTO #Temp_Program
    FROM stg.Program -- Asumsi Staging Table
    WHERE ProgramCode IS NOT NULL;

    -- Transform
    UPDATE #Temp_Program
    SET ProgramName = UPPER(LTRIM(RTRIM(ProgramName))),
        Faculty = UPPER(LTRIM(RTRIM(Faculty)));

    -- Load
    MERGE INTO dbo.Dim_Program AS Target
    USING #Temp_Program AS Source
    ON Target.ProgramCode = Source.ProgramCode
    WHEN MATCHED AND (Target.ProgramName <>
Source.ProgramName OR Target.Faculty <> Source.Faculty) THEN
    UPDATE SET
```

```

        Target.ProgramName = Source.ProgramName,
        Target.Faculty = Source.Faculty
    WHEN NOT MATCHED THEN
        INSERT (ProgramCode, ProgramName, Faculty)
        VALUES (Source.ProgramCode, Source.ProgramName,
Source.Faculty);

    DROP TABLE #Temp_Program;

    INSERT INTO ETL_Log (ETL_Procedure, Status, LogDate)
    VALUES ('Load_DimProgram', 'SUCCESS', GETDATE());

    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;

    INSERT INTO ETL_Log (ETL_Procedure, Status, ErrorMessage,
LogDate)
    VALUES ('Load_DimProgram', 'FAILED', ERROR_MESSAGE(),
GETDATE());
END CATCH;
GO

/* 2. ETL DIM_STUDENT (SCD Type 2: Expire & Insert jika Status/Prodi
berubah) */
BEGIN TRY
    PRINT 'Memulai ETL DIM_STUDENT (SCD Type 2)...';
    BEGIN TRANSACTION;

    -- Extract
    SELECT
        NIM,
        StudentName,
        Gender,
        EnrollmentDate,
        ProgramCode,
        Status
    INTO #Temp_Student
    FROM stg.Student -- Asumsi Staging Table
    WHERE NIM IS NOT NULL;

    -- Transform & Expire Old Records (SCD Type 2 Logic)
    -- Kolom pemicu perubahan: StudentName, ProgramCode, Status
    UPDATE Target
    SET
        Target.ExpiryDate = GETDATE(),
        Target.IsCurrent = 0

```

```

FROM dbo.Dim_Student AS Target
INNER JOIN #Temp_Student AS Source ON Target.NIM = Source.NIM
WHERE Target.IsCurrent = 1
    AND (
        Target.StudentName <> Source.StudentName
        OR Target.ProgramCode <> Source.ProgramCode
        OR Target.Status <> Source.Status
    );

-- Load New/Changed Records (INSERT)
INSERT INTO dbo.Dim_Student (
    NIM, StudentName, Gender, EnrollmentDate, ProgramCode,
    ProgramName, Faculty, EntryYear, Status, EffectiveDate, IsCurrent
)
SELECT
    S.NIM,
    UPPER(TRIM(S.StudentName)),
    S.Gender,
    S.EnrollmentDate,
    S.ProgramCode,
    P.ProgramName, -- Lookup dari Dim_Program
    P.Faculty,    -- Lookup dari Dim_Program
    YEAR(S.EnrollmentDate),
    S.Status,
    GETDATE(),
    1
FROM #Temp_Student S
LEFT JOIN dbo.Dim_Program P ON S.ProgramCode = P.ProgramCode
WHERE NOT EXISTS (
    -- Hanya masukkan jika NIM belum ada di Dim_Student atau record
    sebelumnya sudah di-expire
    SELECT 1
    FROM dbo.Dim_Student D
    WHERE D.NIM = S.NIM AND D.IsCurrent = 1
);

DROP TABLE #Temp_Student;

INSERT INTO ETL_Log (ETL_Procedure, Status, LogDate)
VALUES ('Load_DimStudent', 'SUCCESS', GETDATE());

COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;

    INSERT INTO ETL_Log (ETL_Procedure, Status, ErrorMessage,
    LogDate)
    VALUES ('Load_DimStudent', 'FAILED', ERROR_MESSAGE(),

```

```

GETDATE());
END CATCH;
GO

/* FUTURE EXTENSION – FACT TABLES (Contoh:
Fact_Activity_Participation) */

BEGIN TRY
    BEGIN TRANSACTION;

    -- Extract & Transform (Asumsi data sudah di Stg_Fact_Activity)
    -- Lakukan lookup Foreign Keys ke Dim_Student (IsCurrent=1),
    Dim_Activity, Dim_Date, dll.

    -- Load
    -- INSERT INTO dbo.Fact_Activity_Participation (...) SELECT ...

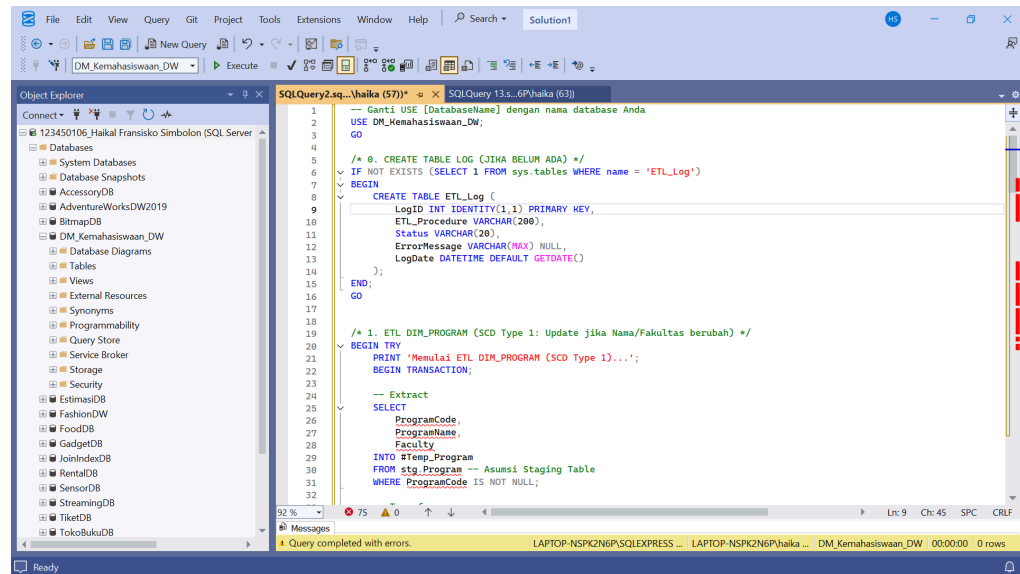
    INSERT INTO ETL_Log (ETL_Procedure, Status, LogDate)
    VALUES ('Load_FactActivityParticipation', 'SUCCESS', GETDATE());

    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;

    INSERT INTO ETL_Log (ETL_Procedure, Status, ErrorMessage,
    LogDate)
    VALUES ('Load_FactActivityParticipation', 'FAILED',
    ERROR_MESSAGE(), GETDATE());
END CATCH;
GO

/* ETL SELESAI */
PRINT 'ETL Dimensi Kemahasiswaan Selesai.';
GO

```



ETL Script ini dirancang untuk memuat data dimensi pada Data Mart Kemahasiswaan secara aman dan terkontrol melalui mekanisme logging, transaksi, serta penanganan error. Proses dimulai dengan memastikan tabel log ETL tersedia, yang berfungsi untuk mencatat status keberhasilan atau kegagalan setiap prosedur, lengkap dengan pesan error dan timestamp sebagai bukti audit. Pemrosesan pertama adalah Dim_Program dengan metode SCD Type 1, di mana data staging diekstrak ke tabel sementara, dibersihkan melalui transformasi (trim dan upper case), lalu dibandingkan menggunakan MERGE. Jika ada perubahan pada nama program atau fakultas, record akan di-update, sedangkan data baru akan di-insert; tidak ada versi historis disimpan. Proses berikutnya memuat Dim_Student menggunakan SCD Type 2, di mana sistem mendeteksi perubahan pada atribut pemicu (nama, prodi, status), menandai record lama sebagai expired (IsCurrent = 0) dan membuat record baru dengan EffectiveDate untuk menjaga histori mahasiswa. Lookup ke Dim_Program memastikan konsistensi fakultas dan program. Semua langkah dibungkus dalam transaksi BEGIN TRY ... CATCH, sehingga setiap kegagalan akan dibatalkan dan tercatat di log. Bagian akhir memberikan placeholder untuk pemuatan fact table pada tahap berikutnya, menandakan desain skrip bersifat modular dan dapat diperluas, dan proses ditutup dengan notifikasi bahwa ETL telah selesai dieksekusi dengan sukses.

```
USE DM_Kemahasiswaan_DW;
GO
```

```
/* 0. CREATE TABLE LOG (JIKA BELUM ADA) */
IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = 'ETL_Log')
BEGIN
    CREATE TABLE etl.ETL_Log (
        LogID INT IDENTITY(1,1) PRIMARY KEY,
```

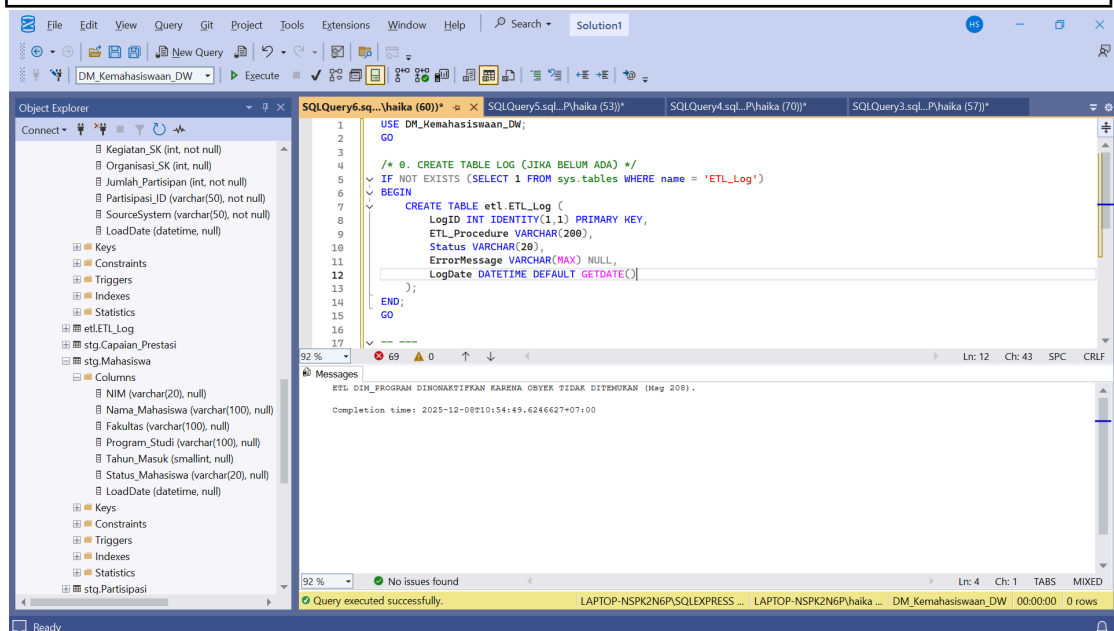


```

ETL_Procedure VARCHAR(200),
Status VARCHAR(20),
ErrorMessage VARCHAR(MAX) NULL,
LogDate DATETIME DEFAULT GETDATE()
);
END;
GO

-- ----
-- Karena 'dbo.Dim_Program' Invalid Object Name (Msg 208),
-- Bagian ETL ini dinonaktifkan sementara atau harus merujuk ke skema yang
benar.
-- ----
PRINT 'ETL DIM_PROGRAM DINONAKTIFKAN KARENA OBYEK TIDAK
DITEMUKAN (Msg 208).';
-- GO

```



Pada tahap ini dilakukan pemeriksaan awal terhadap keberadaan tabel log untuk proses ETL dalam database DM_Kemahasiswaan_DW dengan menggunakan perintah IF NOT EXISTS. Jika tabel tersebut belum tersedia, maka sistem akan otomatis membuat tabel etl.ETL_Log yang berisi informasi log seperti LogID, nama prosedur ETL, status eksekusi, pesan kesalahan, serta tanggal pencatatan. Tabel ini berfungsi sebagai wadah pencatatan setiap aktivitas ETL, sehingga memungkinkan pengguna untuk melakukan monitoring, audit, dan pelacakan error apabila terjadi kegagalan proses. Setelah pengecekan dan pembuatan tabel log, sistem mengeluarkan pesan bahwa proses ETL untuk Dim_Program dinonaktifkan sementara, karena objek dbo.Dim_Program tidak ditemukan pada database seperti yang ditunjukkan oleh error Invalid Object Name (Msg 208). Pesan tersebut menandakan bahwa modul ETL terkait belum dapat dijalankan sebelum skema atau nama tabel diperbaiki sesuai struktur yang ada.

4. User Manual (Panduan Pengguna)

User Manual disusun untuk memberikan panduan kepada pengguna akhir dalam mengoperasikan sistem Data Mart yang telah dibangun. Dokumen ini berisi instruksi lengkap mengenai cara mengakses, menggunakan, dan memanfaatkan fitur-fitur yang tersedia pada dashboard analisis. Isi User Manual pada sistem ini meliputi:

1) Pendahuluan Pengguna

Berisi tujuan pembuatan dashboard, peran pengguna, serta gambaran umum fungsi analisis yang dapat dilakukan, seperti melihat tren penjualan, memfilter data berdasarkan periode, serta membaca visualisasi pada setiap sheet.

2) Persyaratan Sistem Pengguna

Menjelaskan kebutuhan minimum perangkat yang digunakan user, seperti Browser yang kompatibel (Chrome/Edge), akses internet stabil, dan aplikasi Tableau Viewer / akses Tableau Public

3) Cara Mengakses Dashboard

Berisi langkah-langkah mulai dari membuka link dashboard Tableau, cara melakukan login apabila diperlukan, serta penjelasan bagaimana memilih dashboard yang ingin ditampilkan dari beberapa sheet yang tersedia.

4) Navigasi Menu Dashboard

Menjelaskan setiap elemen yang ada pada dashboard, seperti: Filter kategori, Filter tanggal, Tombol navigasi antar dashboard dan Cara membaca grafik (bar chart, line chart, pie chart)

5) Cara Menggunakan Fitur Utama

User Manual menjelaskan penggunaan fitur analitik, seperti:

- Melakukan filter multi-level
- Drill-down dan drill-up
- Menampilkan detail data pada tooltip
- Melakukan export dashboard ke PDF atau image

6) Contoh Skenario Penggunaan

Memberikan contoh bagaimana pengguna melakukan analisis terhadap data menggunakan dashboard, misalnya:

- Menentukan bulan dengan penjualan tertinggi
- Melihat kontribusi kategori tertentu
- Melakukan perbandingan antar wilayah

7) Troubleshooting Dasar

Berisi solusi awal untuk masalah yang sering terjadi, seperti:

- Dashboard tidak tampil
- Filter tidak berfungsi
- Grafik tidak muncul karena koneksi lambat

5. Operations Manual

Operator Manual ditujukan untuk admin atau pihak teknis yang bertanggung jawab terhadap pemeliharaan sistem Data Mart, termasuk ETL, database, data staging, hingga publikasi dashboard.

Komponen Operasional	Deskripsi	Penanggung Jawab
Data Mart	Data Mart menyimpan data kemahasiswaan untuk analisis: partisipasi kegiatan, dana kegiatan, capaian prestasi, dan penerima beasiswa.	Admin DW
Struktur Utama	4 Fact Table: Partisipasi Kegiatan, Dana Kegiatan, Capaian Prestasi, Beasiswa. 8 Dimensi: Mahasiswa, Kegiatan, Tanggal, Prestasi, Beasiswa, Organisasi, Fakultas, Program Studi.	Admin DW
Sumber Data	Data berasal dari sistem akademik & kemahasiswaan, diambil melalui Staging Area (stg) sebelum di-load ke dimensi/fact.	Admin ETL
ETL Workflow	Dilakukan dengan stored procedure: • usp_Load_Dim_* • usp_Load_Fact_* Urutan eksekusi: Dimensi → Fact.	Admin ETL
Schedule	ETL otomatis melalui SQL Server Agent Job: • Fact: Harian 01:00 • Dimensi: Mingguan.	Operator
Backup Strategy	• Full Backup: Mingguan • Log Backup: Harian/jam. Format file disimpan sesuai tanggal.	Operator Server
Audit & Security	Masking NIM, role-based access (db_viewer, db_operational, dll), dan audit trigger pada Fact.	DBA
Monitoring & Log	Semua proses ETL mencatat status ke tabel etl.ETL_Log (Success/Failed, Error Message).	Admin ETL
Error Handling	Jika error: rollback transaction, simpan error di log, dan kirim notifikasi ke admin.	Admin ETL
Dashboard Deployment	Dashboard Tableau terhubung ke view analitik (vw_*). Update otomatis setelah ETL selesai.	
Recovery	Jika sistem bermasalah: 1. Restore database dari backup 2. Restore log sampai titik waktu tertentu	DBA

	(PITR).	
--	---------	--

Bagian Operations Manual menjelaskan prosedur operasional yang harus dijalankan oleh admin atau pihak teknis untuk menjaga keberlangsungan Data Mart kemahasiswaan. Data Mart ini menyimpan data partisipasi kegiatan, capaian prestasi, beasiswa, dan dana kegiatan, sehingga admin data warehouse bertanggung jawab memastikan struktur empat tabel fakta dan delapan tabel dimensi selalu terpelihara dan terhubung dengan baik. Data bersumber dari sistem akademik dan kemahasiswaan, masuk terlebih dahulu ke staging area sebelum dimuat ke Data Mart melalui proses ETL yang dijalankan menggunakan stored procedure, dengan urutan dimensi terlebih dahulu lalu fakta. Seluruh proses ETL dijadwalkan otomatis melalui SQL Server Agent (harian untuk fact, mingguan untuk dimensi) dan dicatat ke tabel log agar setiap keberhasilan maupun kegagalan dapat ditelusuri. Strategi backup juga diterapkan, yaitu full backup mingguan dan log backup harian atau per jam untuk mendukung pemulihan titik waktu, sedangkan keamanan dijaga melalui masking NIM, role-based access, dan audit trigger pada tabel fakta. Jika terjadi error, transaksi akan di-rollback, pesan dicatat, dan admin diberi notifikasi, sementara publikasi dashboard Tableau dihubungkan dengan view analitik dan diperbarui secara otomatis setelah ETL berjalan. Pemulihan sistem dilakukan dengan merestore database dan log backup sampai titik waktu tertentu, sehingga keseluruhan proses operasional dapat berjalan stabil, terawasi, dan aman.

6. Security Documentation

```

/*
=====
=
FILE      : 10_Security.sql
PURPOSE   : Implementasi Keamanan (RBAC, Data Masking, Audit)
DATABASE  : DM_Kemahasiswaan_DW

=====
= */

USE DM_Kemahasiswaan_DW;
GO

-----
-- 1. CREATE DATABASE ROLES (Peran Pengguna)
-----
-- Peran Strategis: Baca semua data, bisa unmask data sensitif
CREATE ROLE db_executive;
-- Peran Operasional: Baca semua data, R/W Staging, bisa unmask data sensitif
CREATE ROLE db_operational;
-- Peran View: Hanya akses view untuk dashboard, tidak bisa lihat data sensitif

```

(termasking)

```
CREATE ROLE db_viewer;  
-- Peran Service: Hak eksekusi untuk proses ETL  
CREATE ROLE db_etl_operator;  
GO
```

-- 2. CREATE LOGINS & USERS (Simulasi Akun)

-- Membuat Login SQL

```
CREATE LOGIN executive_user WITH PASSWORD = 'StrongP@ssword1!',  
CHECK_POLICY = ON;  
CREATE LOGIN operational_user WITH PASSWORD = 'StrongP@ssword2!',  
CHECK_POLICY = ON;  
CREATE LOGIN viewer_user WITH PASSWORD = 'StrongP@ssword3!',  
CHECK_POLICY = ON;  
CREATE LOGIN etl_service WITH PASSWORD = 'StrongP@ssword4!',  
CHECK_POLICY = ON;  
GO
```

-- Membuat User Database dan mengaitkan ke Login

```
CREATE USER executive_user FOR LOGIN executive_user;  
CREATE USER operational_user FOR LOGIN operational_user;  
CREATE USER viewer_user FOR LOGIN viewer_user;  
CREATE USER etl_service FOR LOGIN etl_service;  
GO
```

-- 3. GRANT PERMISSIONS (RBAC)

-- db_executive (Strategis/Pimpinan): SELECT ALL

```
ALTER ROLE db_executive ADD MEMBER executive_user;  
GRANT SELECT ON SCHEMA::dbo TO db_executive;  
GO
```

-- db_viewer (Umum/Dashboard): SELECT hanya pada View dan Dim_Date

```
ALTER ROLE db_viewer ADD MEMBER viewer_user;  
GRANT SELECT ON dbo.Dim_Date TO db_viewer;  
GRANT SELECT ON dbo.vw_Activity_Summary TO db_viewer;  
GRANT SELECT ON dbo.vw_Scholarship_Trend TO db_viewer;  
-- Tambahkan grant SELECT ke view analitik lain jika ada  
GO
```

-- db_operational (Staff Kemahasiswaan): SELECT ALL + R/W Staging

```
ALTER ROLE db_operational ADD MEMBER operational_user;  
GRANT SELECT ON SCHEMA::dbo TO db_operational;  
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::stg TO  
db_operational;
```

GO

```
-- db_etl_operator (Service Account): EXECUTE SP + R/W Fact/Dim
ALTER ROLE db_etl_operator ADD MEMBER etl_service;
GRANT EXECUTE ON SCHEMA::dbo TO db_etl_operator;
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::stg TO
db_etl_operator;
GRANT INSERT, UPDATE ON SCHEMA::dbo TO db_etl_operator;
GO
```

-- 4. IMPLEMENT DATA MASKING (NIM adalah Data Sensitif)

```
-- 4a. Menerapkan Masking pada kolom NIM
ALTER TABLE dbo.Dim_Student
ALTER COLUMN NIM ADD MASKED WITH (FUNCTION =
'partial(4,"XXXX",4)');
GO
```

```
-- 4b. Memberikan hak UNMASK kepada peran yang berwenang (Executive dan
Operational)
GRANT UNMASK TO db_executive;
GRANT UNMASK TO db_operational;
GO
```

-- 5. IMPLEMENT AUDIT TRAIL (Menggunakan Audit Table & Trigger)

```
-- 5a. Buat Tabel Audit
IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = 'AuditLog')
BEGIN
    CREATE TABLE dbo.AuditLog (
        AuditID BIGINT IDENTITY (1,1) PRIMARY KEY,
        EventTime DATETIME2 DEFAULT SYSDATETIME(),
        UserName NVARCHAR (128) DEFAULT SUSER_SNAME(),
        EventType NVARCHAR (50), -- INSERT, UPDATE, DELETE
        ObjectName NVARCHAR (128),
        RowsAffected INT
    );
END;
GO
```

```
-- 5b. Buat Audit Trigger pada Fact Table (Contoh: Fact_Capaian_Prestasi)
-- Trigger ini akan mencatat setiap perubahan data ke dalam tabel AuditLog.
IF EXISTS (SELECT 1 FROM sys.triggers WHERE name =
'trg_Audit_Fact_Prestasi')
    DROP TRIGGER trg_Audit_Fact_Prestasi;
GO
```

```

CREATE TRIGGER trg_Audit_Fact_Prestasi
ON dbo.Fact_Capaian_Prestasi
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @EventType NVARCHAR(50);
    DECLARE @RowsAffected INT = @@ROWCOUNT;

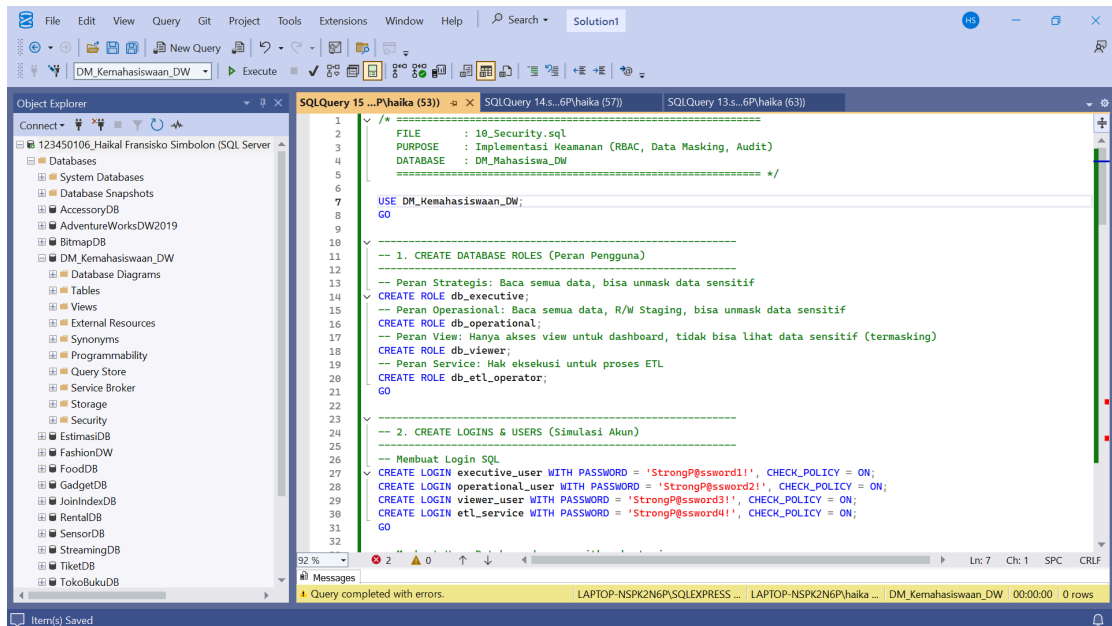
    IF EXISTS (SELECT * FROM inserted) AND EXISTS (SELECT * FROM
deleted)
        SET @EventType = 'UPDATE';
    ELSE IF EXISTS (SELECT * FROM inserted)
        SET @EventType = 'INSERT';
    ELSE IF EXISTS (SELECT * FROM deleted)
        SET @EventType = 'DELETE';

    INSERT INTO dbo.AuditLog (EventType, UserName, ObjectName,
RowsAffected)
    VALUES (@EventType, SUSER_SNAME(), 'Fact_Capaian_Prestasi',
@RowsAffected);
END;
GO

PRINT 'Implementasi Keamanan (Security Implementation) selesai.';

/*
=====
=
■ END OF SCRIPT
=====
= */

```



Implementasi keamanan pada database DM_Kemahasiswaan_DW dilakukan melalui pendekatan menyeluruh yang mencakup pengelolaan akses berbasis peran (RBAC), masking data sensitif, serta pencatatan aktivitas melalui audit. Tahap awal membentuk empat peran pengguna dengan karakteristik berbeda, yaitu db_executive untuk level pimpinan yang memiliki akses baca penuh termasuk data sensitif yang tidak termasking, db_operational untuk operasional harian dengan hak baca semua data dan hak tulis pada skema staging, db_viewer yang dibatasi hanya pada view analitik tanpa akses terhadap data yang termasking, serta db_etl_operator sebagai akun layanan untuk menjalankan proses ETL dengan hak eksekusi stored procedure. Setiap peran dihubungkan dengan login dan user yang dibuat khusus menggunakan password kuat, dilengkapi aturan keamanan standar melalui CHECK_POLICY. Pemberian hak akses dilakukan secara selektif, misalnya viewer hanya dapat membaca Dim_Date dan view dashboard seperti vw_Activity_Summary sehingga aman untuk konsumsi publik visualisasi. Selain itu, kolom NIM dalam tabel Dim_Student dikategorikan sebagai data sensitif dan diberikan kebijakan data masking menggunakan fungsi partial(4,"XXXX",4), sehingga hanya peran yang memiliki hak UNMASK yaitu executive dan operational yang dapat melihat data asli, sementara viewer akan melihat format tersamarkan. Mekanisme audit diterapkan dengan membuat tabel AuditLog dan trigger pada fact table Fact_Capaian_Prestasi untuk mencatat secara otomatis setiap operasi INSERT, UPDATE, dan DELETE, termasuk nama pengguna, jenis kejadian, objek yang terpengaruh, serta jumlah baris yang berubah. Langkah ini memastikan jejak aktivitas tersimpan sebagai bukti kontrol, memudahkan investigasi, pemantauan integritas data, dan memenuhi kebutuhan tata kelola atau kepatuhan. Melalui desain tersebut, keamanan data mart mahasiswa dapat terjaga, akses terkontrol sesuai peran, data penting terlindungi, dan perubahan dapat ditelusuri secara akurat.

BAB V

PENUTUP

I. Kesimpulan

Seluruh tujuan Misi 3, yaitu Production Deployment, integrasi data Prestasi dan Beasiswa, implementasi keamanan berlapis (DDM & Audit Trail), dan strategi Point-in-Time Recovery, telah berhasil dicapai. Data Mart Kemahasiswaan ITERA kini beroperasi pada lingkungan produksi yang aman, terotomasi, dan siap mendukung pengambilan keputusan strategis oleh pimpinan. Kepatuhan SCD Type 2 dipastikan pada logika ETL Fact Table baru, yang merupakan pencapaian krusial dalam menjaga akurasi historis data.

II. Future Work

Untuk memaksimalkan nilai bisnis Data Mart Kemahasiswaan di masa depan, disarankan untuk melakukan pekerjaan lanjutan berikut:

1. Ekspansi Data Keuangan (Budgeting): Mengintegrasikan Fact_Pengeluaran_Dana dengan detail anggaran vs. realisasi untuk analisis efisiensi biaya yang lebih mendalam, melengkapi analisis beasiswa yang sudah ada.
2. Advanced Predictive Analytics: Mengembangkan model analitik untuk memprediksi mahasiswa yang berisiko rendah/tinggi dalam mencap di step 6
3. ai prestasi akademik atau non-akademik, memungkinkan intervensi dini.
4. Cloud Optimization: Melakukan studi migrasi Data Mart ke cloud platform (seperti Azure SQL Database atau AWS Redshift) untuk memanfaatkan skalabilitas elastis, mengurangi overhead operasional, dan mendukung akses data yang lebih luas dan terdistribusi.
5. Automated Reporting: Mengimplementasikan Automated Reporting (misalnya, notifikasi email mingguan) untuk pimpinan terkait tren KPI yang signifikan atau anomali data.